

Full Report

AI-Assisted API Design

Reducing API design efforts without sacrificing developer experience

December 2025

James Higginbotham
james@launchany.com



Table of Contents

Executive Summary	Page 3	3 AI-Powered API Design In Action	Page 25
Inside the Report		An Annotated API Design Walkthrough	
1 Introduction	Page 6	4 Choosing Your AI Assistant	Page 62
Why Is Modern API Design Still Difficult?		LLM Capability Matrix	
2 Your Time Multiplier	Page 12	5 Taking the First Step	Page 76
Using AI as an API Design Assistant		Try it Yourself	

Executive Summary

Making the Strategic Shift to AI as an API Design Coach

The traditional API design process today is bottlenecked by limited time and budgets, forcing teams to choose between delivery speed and design quality. This "rush to code" introduces significant technical debt and fails to support the agility required for modern AI strategies.

This report outlines a transformational shift: leveraging Large Language Models (LLMs) not merely as code assistants, but as API Design Coaches. By pairing the ADDR process with AI, teams can move from manual drafting to strategic oversight.

Key Findings

- **Drastic Time Reduction:** AI-assisted workflows reduce API design time by 60% (and up to 20x faster in some phases) while improving consistency.
- **High-Fidelity Artifacts:** Critical deliverables—such as Job Stories and OpenAPI specifications—can now be auto-generated with >95% accuracy.
- **Human-Centric Design:** AI handles the heavy lifting of artifact creation, allowing the API designers to focus on aligning business goals with user outcomes. (“AI in the loop”)
- **The Competitive Edge:** Organizations that adopt AI-assisted API design in 2026 will outpace competitors not by writing code faster, but by making better architectural decisions faster..

Inside the Report

Who should read this report?

Practitioners looking for a way to speed their API design efforts without sacrificing a great developer experience.

API program leaders tired of low reuse and duplicate API efforts.

AI transformation leaders that wish to expand their AI value creation beyond GenAI co-development.

After reading this report, you will be able to:

- Identify the challenges facing API designers in today's fast-paced, AI-driven landscape.
- Apply the ADDR API design process within the context of an AI-assisted session that acts as an API design coach.
- Experience a walkthrough of an API design session that includes prompting tips, guardrails, and example outputs.
- Select an LLM model that best fits your needs and is trained to solve API design challenges.
- Explore additional resources to get you started quickly with an AI-assisted API design coach.

Get the most out of this report

Get the Prompt Starters

Prompt Starters for AI-Assisted API Design

Example prompts are provided below to help you get started with your own AI-assisted API design coach session. Of course, each starter prompt will need to be supplemented by your own prompting as you progress through each phase to direct the model to change, remove, or add anything that may be missing.

Tips and Guide Rails

- **Separate chat sessions when needed.** For some models, you may need to separate each phase into a separate chat session. In that case, export the artifacts from each previous chat session and import all artifacts into the next chat session. This will reduce hallucinations or sessions that turn sloppy or run amok.
- **Prefer iterative work over one-shot design sessions.** The larger the scope of your project, the more bounded API contexts you may have. When you have more than one, limit the model to operating on the first boundary in its list (or a boundary of your choosing) before proceeding to each boundary in turn. Failing to take this approach often yields poor results or incomplete responses.
- **Ask the model to generate your style guide prompt.** When you are ready to turn your ADDR artifacts into an API design, launch a separate chat session and ask the model to generate a prompt from your style guide. This will help you to align your designs to your organization's API style guide during design and reduce the amount of API design review that is required.

New Chat Session Prompt

This prompt is critical to establish the role of the model, the steps we will take, and that we will focus on requirements prior to starting the ADDR process. Reviewing requirements first helps to spot gaps and also helps to drive the API design effort.

```
You are an API architect that needs to translate requirements into an API design. First, I will ask you to review an API design.

We will be designing a bookstore API.

However, before we start ADDR, I need some help refining the requirements. Once we validate that the requirements are clear,

I will provide initial requirements next.
```

After reading the report, be sure to go to our GitHub repository where you can find provide starter prompts, guiderails, and tips.



<https://bit.ly/addr-ai-prompts>

Plus, check out the additional resources, including the ADDR book and training materials, at the end of the report.



1 Introduction

Why Is Modern API Design Still Difficult?

Let's Explore:

- The Challenges of API Design
- Common API Design Challenges
- Can We Do Better?
- 3 Steps to Get Started

Limited Time. Limited Budgets.

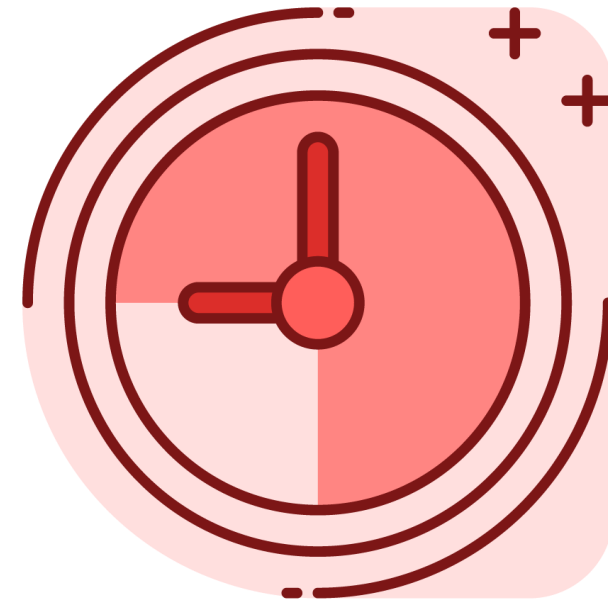
The last time you tried to design a great API, how did it go?

You are being told to do more with less. Great API requires time, but your time to design APIs is shrinking every day.

The last time you tried to design a great API, you were stuck in so many meetings. By the end of the meetings, things still weren't clear.

Yet, without a clear set of requirements, last-minute changes will be required. This will introduce bugs and technical debt that will be hard to address in the future.

The challenges of great API design can be summarized as limited time and limited budget.



Speeding Up the API Design Process

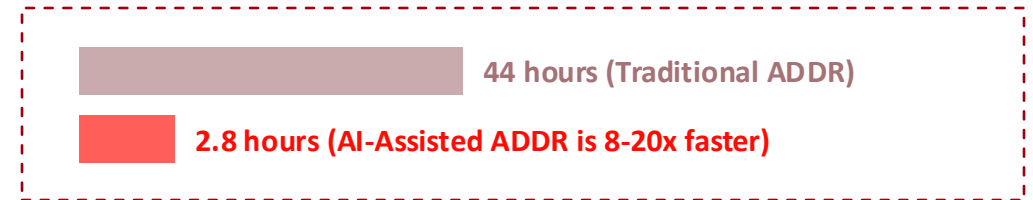
Spend less time typing and more time thinking.

For over a decade, I have been exploring ways to confidently and thoughtfully design APIs that developers will love while ensuring that they are reusable across today's emerging opportunities.

LLMs are now becoming mature enough that API designers can leverage them to do more than write code. They can use them as an API design assistant.

What we have now found is that teams are able to leverage a repeatable, scalable API design process in conjunction with today's LLM capabilities.

The result: More time for architectural decisions and faster feedback loops due to reduced manual effort.



AI-Assisted API design allows teams to spend less time typing and more time thinking, while aligning business decisions with reusable, AI-enabled APIs.

Speeding Up the API Design Process

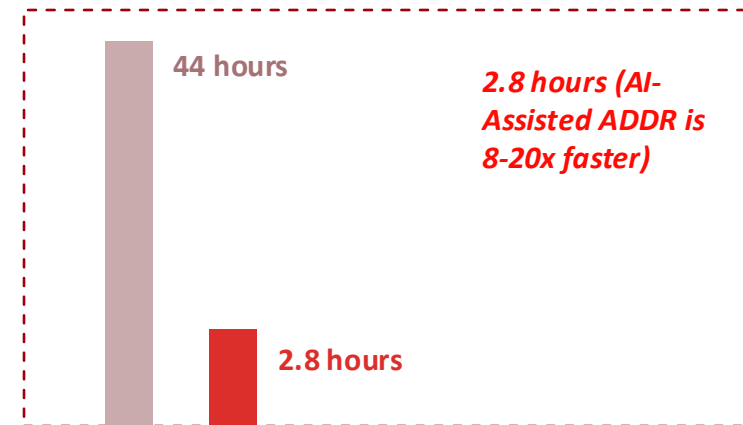
Spend less time typing and more time thinking.

For over a decade, I have been exploring ways to confidently and thoughtfully design APIs that developers will love while ensuring that they are reusable across today's emerging opportunities.

LLMs are now becoming mature enough that API designers can leverage them to do more than write code. They can use them as an API design assistant.

What we have now found is that teams are able to leverage a repeatable, scalable API design process in conjunction with today's LLM capabilities.

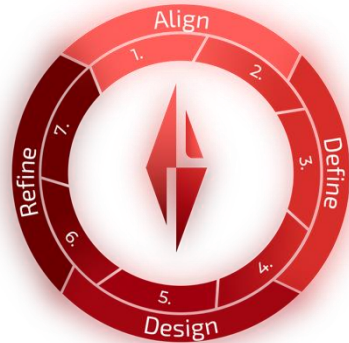
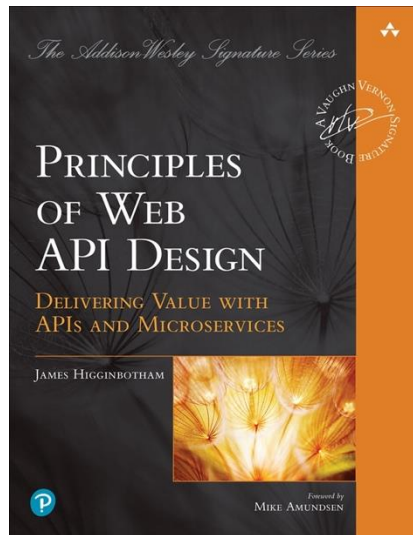
The result: More time for architectural decisions and faster feedback loops due to reduced manual effort.



AI-Assisted API design allows teams to spend less time typing and more time thinking, while aligning business decisions with reusable, AI-enabled APIs.

The ADDR Process at a Glance

The four phases of the ADDR process transform requirements into a business-aligned API design.



Align – “Outcomes before endpoints”

Ensure alignment between your business architecture, the desired outcomes of your users, and your APIs.

Define – “Model first to build fast”

Map desired outcomes to an API profile that will manage resources and support the intended workflow

Design – “Collaborate around the contract”

Transform your API profile into a high-level API design that complies with your style guide.

Refine – “Feedback loops create confidence”

Incorporate feedback to finalize your production-ready decisions and documentation.

4 Steps to Faster API Design

Leverage today's LLM capabilities to speed API design without sacrificing quality.

1. Learn about the ADDR API design process.

If you aren't familiar with the ADDR API design process, we can help. This report provides a brief overview of the process and the artifacts that help you go from requirements to a transformative API that delivers desired outcomes for humans and AI.

2. Explore how LLMs accelerate API design.

Find out how LLMs can help to accelerate the API design process, from requirements gathering to defining a clear API specification for REST, GraphQL, and gRPC-based APIs.

3. Select an LLM as your AI Design Coach.

Not all LLMs are best for API design. We did the hard work for you, evaluating the most popular LLMs that support API design efforts. No external tool requirements or add-ons are required, making it easy to get started.

4. Start Designing Great APIs.

Finally, start designing APIs with an LLM-based API design coach. When ready, hand off your API design specification to your engineering teams for build out using AI code-development with tools such as GitHub Copilot or Claude Code.

Ready? Let's get started.



2 Your Time Multiplier

Using AI as an API Design Assistant

Let's Explore:

- GenAI is for more than code
- Why Early GenAI Wasn't Ready
- The ADDR Process at a Glance
- How LLMs accelerate the ADDR process

GenAI is For More Than Code

Too many organizations are missing out on what AI can really do.

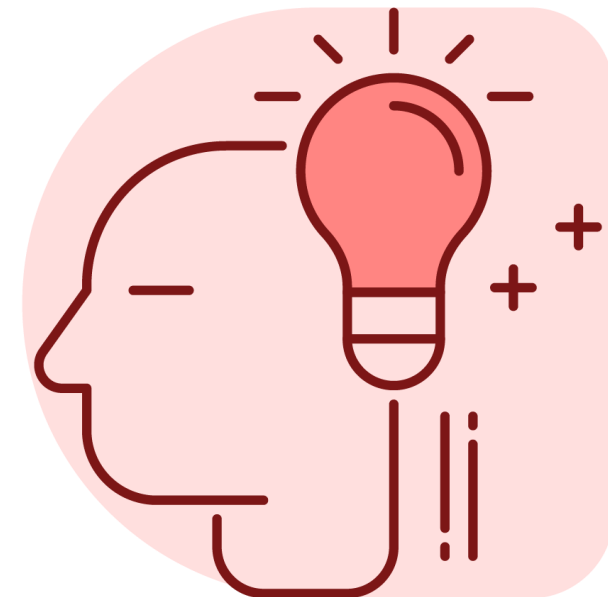
Most organizations today see GenAI as a tool to write more code faster. While this provides short-term gains, **the full potential of generative AI hasn't been realized.**

As GenAI evolves, new capabilities are beginning to emerge.

We explored whether an LLM could act as an API design coach. **The results were impactful.**

AI-in-the-loop is the best way to describe this approach. Experts leverage prompts and LLM automation to accelerate their API design efforts for critical business systems.

Let's explore what has changed that allows GenAI to assist in the API design process. We will then explore the ADDR process and how GenAI can help.



Why Early GenAI Wasn't Ready

Early exploration of API models weren't ready to coach someone through an API design process. This has recently changed.

Context windows were previously too limiting.

Now, better prompt engineering can overcome this with recent AI models. This results in reduced hallucinations while avoiding poor or incomplete API designs.

Former LLM models couldn't break down complex workflows.

We now have advanced models that can decompose complex problems into an executable plan. The plan guides practitioners effectively through the process.

Response outputs were limited to text-based formats.

Recent AI models have added new ways to generate output formats than in the past. This reduces the manual work required for generating artifacts, incorporating feedback, and delivering a great API specification.



Do We Need API Design with AI and Vibe-Coding?

Where this report fits in the AI transformation journey.

Vibe Coding emerged in early 2025 as a new development style that refers to an intent-based approach to development that uses a **prompt, run, fix methodology**.

It is commonly used for **rapid prototyping, creating disposable tools, and non-critical applications**.

Business Critical AI refers to the use of AI-augmented engineering that uses a **spec, prompt, test, human review methodology** to produce reliable, deterministic code for critical systems.

It is commonly used when **reliability, explainability, and guardrail enforcement** is paramount.



Vibe Coding help organizations find the **right idea fast**.

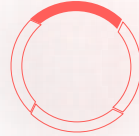
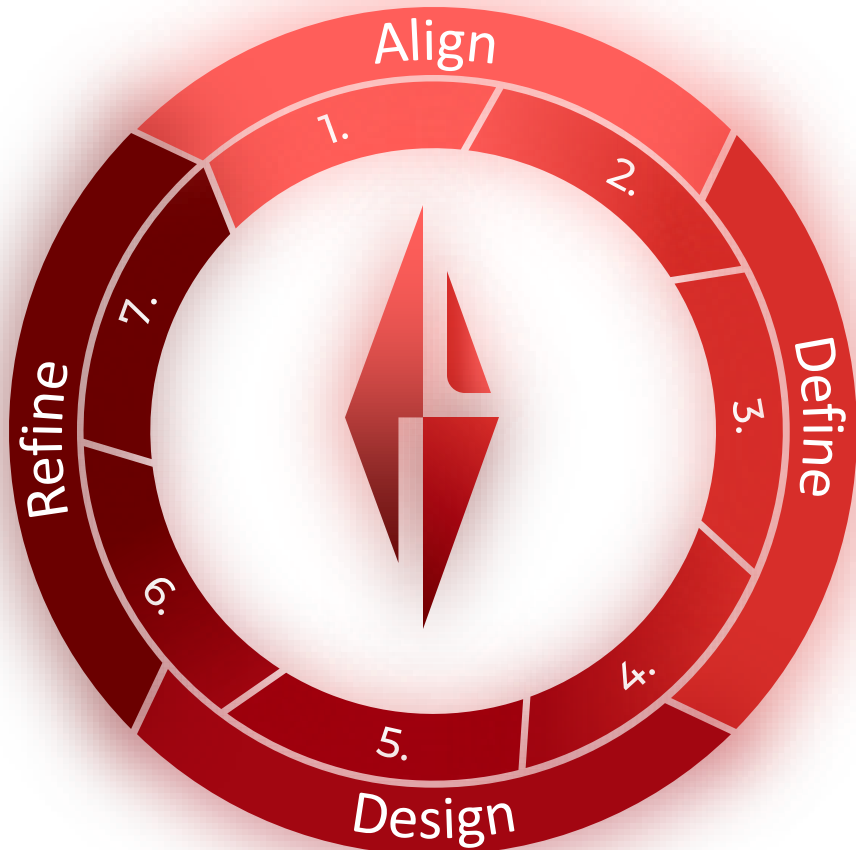


Business Critical AI helps to **scale the idea safely**.

AI-assisted API design speeds the process of transforming insights into business-critical systems.

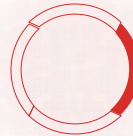
The ADDR Process at a Glance

The four phases of the ADDR process transform requirements into a business-aligned API design.



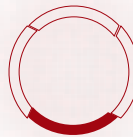
Align – “Outcomes before endpoints”

Ensure alignment between your business architecture, the desired outcomes of your users, and your APIs.



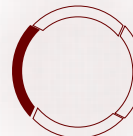
Define – “Model first to build fast”

Map desired outcomes to an API profile that will manage resources and support the intended workflow



Design – “Collaborate around the contract”

Transform your API profile into a high-level API design that complies with your style guide.

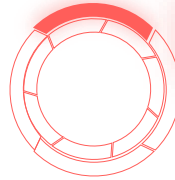
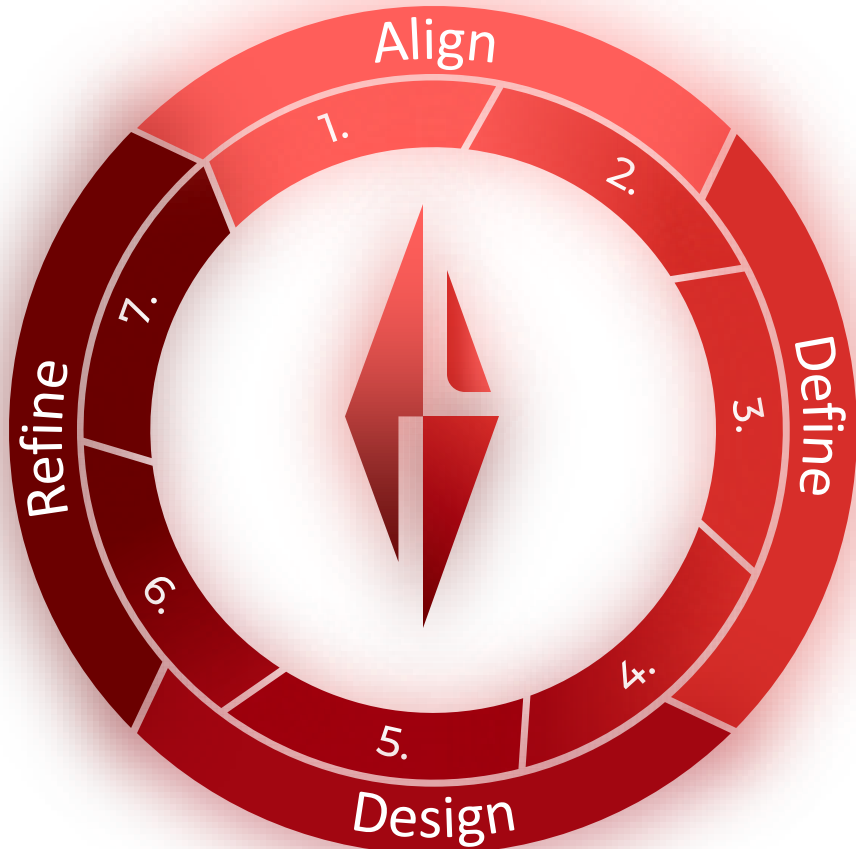


Refine – “Feedback loops create confidence”

Incorporate feedback to finalize your production-ready decisions and documentation.

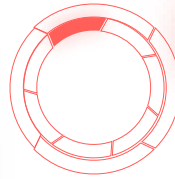
The ADDR Align Phase at a Glance

“Outcomes before endpoints”



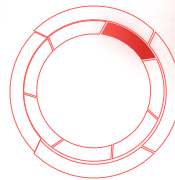
Align

Ensure alignment between your business architecture, the desired outcomes of your users, and your APIs.



1. Identify Digital Capabilities

Capture jobs-to-be-done (JTBD) to better understand any problems/situations, the job, and the desired outcome. This will frame all future API design work.



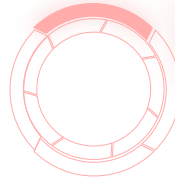
2. Capture Activity Steps

Break down the jobs into activity steps, including any specific workflow steps required to be completed.

AI Assistance in the Align Phase

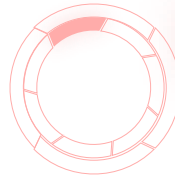
How to leverage LLMs in the Align Phase of ADDR

- 1 Use LLMs to dig deeper into requirements, look for gaps, and surface missing outcomes.
- 2 Let the LLM turn your insights into job stories that capture problems and desired outcomes your API will solve.
- 3 Break your job stories into activity steps with the help of the LLM. Look for gaps that need to be addressed.



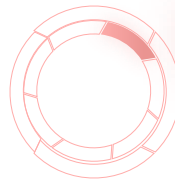
Align

Ensure alignment between your business architecture, the desired outcomes of your users, and your APIs.



1. Identify Digital Capabilities

Capture jobs-to-be-done (JTBD) to better understand any problems/situations, the job, and the desired outcome. This will frame all future API design work.

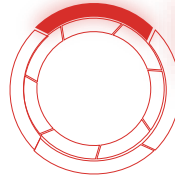
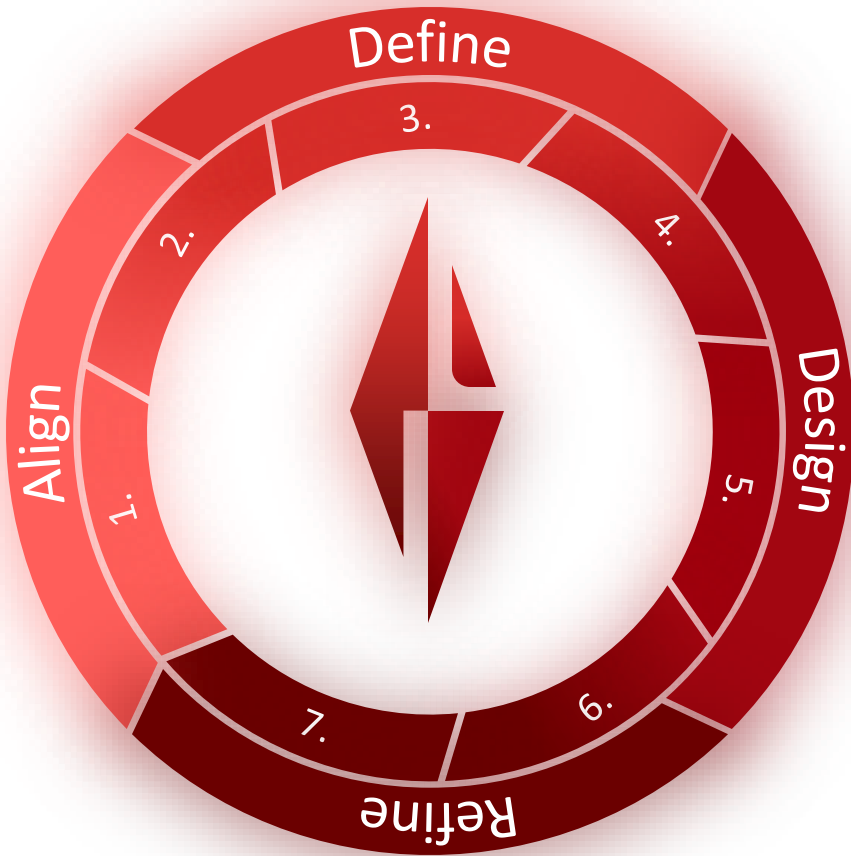


2. Capture Activity Steps

Break down the jobs into activity steps, including any specific workflow steps required to be completed.

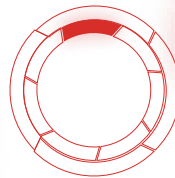
The ADDR Define Phase at a Glance

“Model first to build fast”



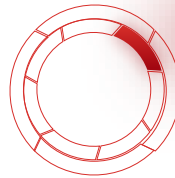
Define

Map desired outcomes to an API profile that will manage resources and support the intended workflow



3. Identify API Ecosystem Boundaries

Assess how the API will be used and what digital ecosystems it will belong.

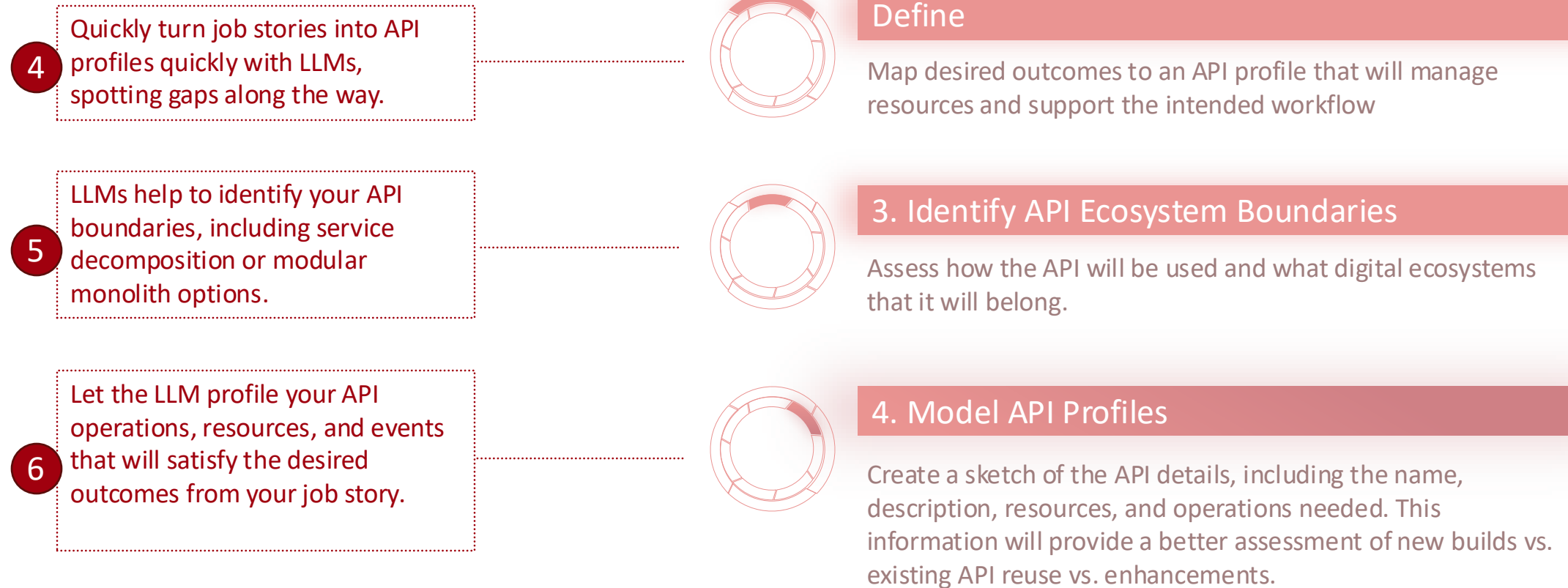


4. Model API Profiles

Create a sketch of the API details, including the name, description, resources, and operations needed. This information will provide a better assessment of new builds vs. existing API reuse vs. enhancements.

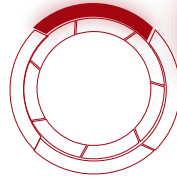
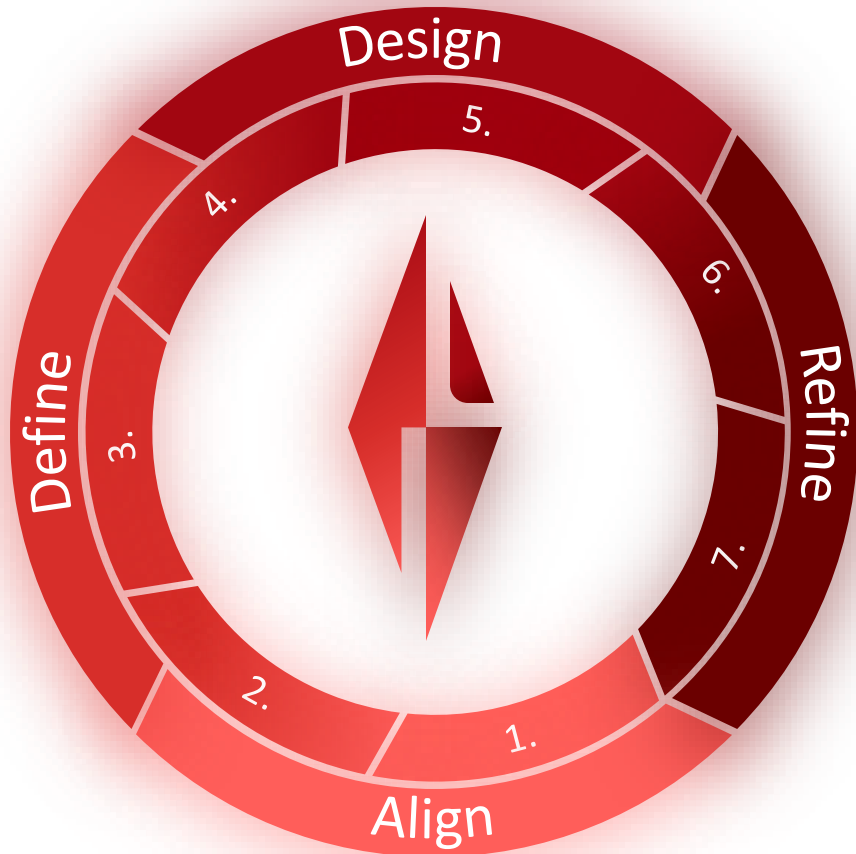
AI Assistance in the Define Phase

How to leverage LLMs in the Define Phase of ADDR



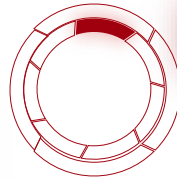
The ADDR Design Phase at a Glance

“Collaborate around the contract”



Design

Transform your API profile into a high-level API design that enables feedback to be incorporated quickly.



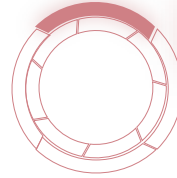
5. High-level Design

Select one or more API design styles that fit the needs of your API consumer, apply style guide preferences, and produce a high-level API design for feedback.

AI Assistance in the Design Phase

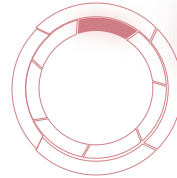
How to leverage LLMs in the Design Phase of ADDR

- 7 Import your API style guidelines into the LLM and avoid dependence on a linter during design.
- 8 Select the best API design style(s) that your API will expose.
- 9 Then, transform your API profiles into a high-level API design in seconds using an LLM.



Design

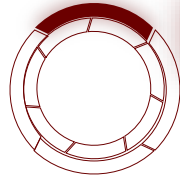
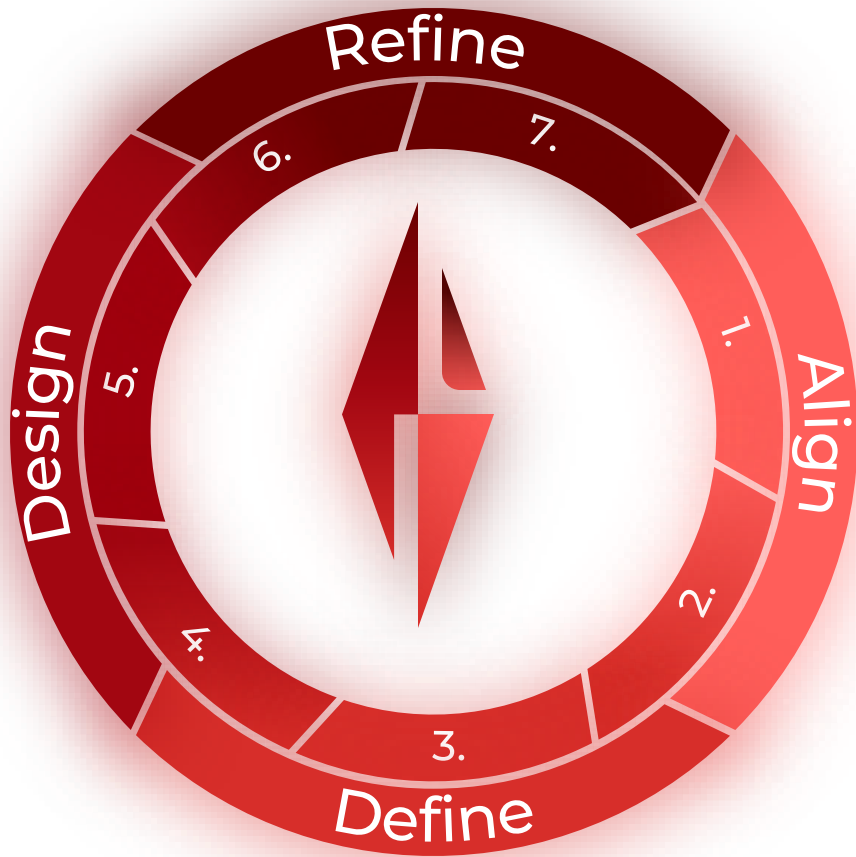
Transform your API profile into a high-level API design that enables feedback to be incorporated quickly.



5. High-level Design

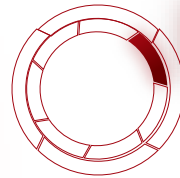
Select one or more API design styles that fit the needs of your API consumer, apply style guide preferences, and produce a high-level API design for feedback.

The ADDR Refine Phase at a Glance



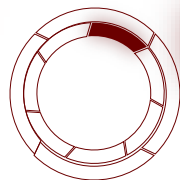
Refine

Finalize your production-ready decisions and documentation based on additional feedback.



6. Refine the Design

Expand the artifacts to communicate more complex use cases using README-based scenarios, identify any gaps, surface any additional insights, and revise the API design now, when the cost of change is lower.



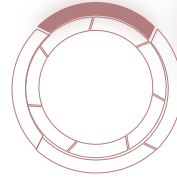
7. Document the API

Finalize the documentation that will be used by API consumers.

AI Assistance in the Refine Phase

How to leverage LLMs in the Refine Phase of ADDR

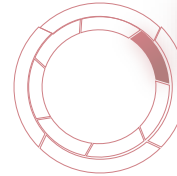
10 Let LLMs take care of the most time-consuming part of API ownership - documentation.



Refine

Finalize your production-ready decisions and documentation based on additional feedback.

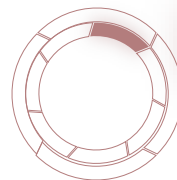
11 In just a few minutes, generate and deliver valuable artifacts that facilitate fast feedback using LLMs.



6. Refine the Design

Expand the artifacts to communicate more complex use cases using README-based scenarios, identify any gaps, surface any additional insights, and revise the API design now, when the cost of change is lower.

12 Produce human and machine-ready documentation for your API based on your LLM session and other artifacts.



7. Document the API

Finalize the documentation that will be used by API consumers.



3 AI-Powered API Design In Action

An Annotated API Design Walkthrough

Let's Explore:

- See a step-by-step walkthrough of using LLMs to deliver an API design
- Identify potential issues and how to overcome them
- Explore prompt engineering tips for effective API design with LLMs

About the Walkthrough

The scope of the walkthrough example.

Bookstore Shopping API

We are designing an ecommerce bookstore API. For this example, we will constrain the API design work to shopping and order placement.



We will explore how to refine your requirements, followed by the ADDR API design process :

1. **Requirements clarification** – Preparing the requirements for ADDR.
2. **ADDR Align Phase** – Turning requirements into job stories + activities.
3. **ADDR Define Phase** – Identifying the API(s) with scope + operations.
4. **ADDR Design Phase** – Producing high-level API design artifacts.
5. **ADDR Refine Phase** – Generating artifacts for fast feedback+ docs.

Understanding the Annotated LLM Interactions

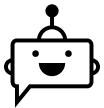
How to get the most out of this walkthrough.

There are two personas involved in the walkthrough:



Human Prompt Engineer

Enters a prompt into the chat session



LLM Response

Performs the work and responds back in the chat session

These two personas will interact via an LLM chat session. Below is an example of how it will look in the walkthrough:



Prompt text provided by the human operator



Response text from the LLM



Prompt text provided by the human operator

#



Response text from the LLM

Explanation and insights to get the most out of your AI-assisted design session.



Reviewing the Requirements

Leverage the LLM to identify gaps or unknowns in your specification.

Coach's Notes

Human Prompt Engineer

You are an API architect that needs to translate requirements into an API design. First, I will ask you to review and help me to refine the requirements. Then we will use ADDR for the API design first methodology.

We will be designing a bookstore API.

However, before we start ADDR, I need some help refining the requirements. Once we validate that the requirements are sufficient, then we will start the ADDR process.

Here are the initial requirements:

- * Online ecommerce store that only sells books
- * Called JSON's Bookstore
- * Must design a series of APIs to support online commerce that includes shopping and checkout capabilities
- * We will also need order fulfillment, order tracking, inventory management, and catalog management capabilities in the future
- * The API also needs to support integration with partners and customers.
- * The API will support a web interface, mobile app, and third-party API consumers
- * Initial Scope: Design APIs to support online shopping and checkout.
- * Future Scope: Order fulfillment, order tracking, inventory management, and catalog management.

Let's hold off on non-functional and technical requirements and focus on the business requirements for now.

Ask me up to 5 questions that will help to expand and clarify the requirements before we proceed with API design. You may ask two more sets of 5 questions each, if they will help you. If you have everything you need, you may stop asking questions prior to the end of the three sets of questions.

Once all questions have been answered, summarize the business requirements. however, before we start ADDR, I need some help refining the requirements. Once we validate that the requirements are sufficient, then we will start the ADDR process.

I will provide initial requirements below:

<truncated for space>.

1

Establish the role of the LLM, along with the use of ADDR for the API design process.

First, we start with a clear set of requirements based on what is currently known. We then ask that the LLM to review and ask clarifying questions.

In this example, we permit the LLM to ask 3 sets of 5 questions each. This will help us dig a bit deeper into the problem to ensure we have a more complete set of requirements from which to design our API.



Reviewing the Requirements

Leverage the LLM to identify gaps or unknowns in your specification.

Coach's Notes



LLM Response

Of course. Clarifying business requirements is the crucial first step in the ADDR methodology. Here are five questions to help refine the requirements for the JSON's Bookstore API.

Clarifying Questions: Set 1

1. **User Roles & Access:** What are the different types of users that will interact with the shopping and checkout process? For example, can a customer check out as a guest, or must they create a registered account first?
2. **The "Book" Entity:** Besides title and author, what are the key attributes that define a book in the catalog? For instance, what information is critical for a customer to see before adding a book to their cart (e.g., ISBN, genre, cover image, synopsis, user ratings)?
3. **Product Discovery:** How do customers typically find books they want to purchase? Should the API primarily support searching by specific fields like title or ISBN, or is browsing by categories, new releases, or curated best-seller lists just as important?
4. **Checkout Flow:** Could you walk me through the specific steps of the checkout process from the customer's perspective? For example, what information is collected (shipping address, payment details), what payment methods will be supported initially, and are there features like applying discount codes?
5. **Order Confirmation:** Once an order is successfully placed, what specific information constitutes an "order confirmation" for the customer? Are we providing just an order ID, or a full summary including item details, total cost, and an estimated delivery date?



Answers:

1. Customers may check out as a guest or may register for an account/sign in
2. Title, author, ISBN, genre, format, price, language, plus previews, reviews, ratings, and recommendations
3. Search & filter by book/author details, price range, and format
4. For the workflow, use the following: <truncated for space>
5. Order ID



<LLM asks the second and third set of questions, truncated for space>

2

The LLM will analyze your requirements and determine what kind of questions need to be asked.

These questions may be answered by you, or it may provide a concise way to gather requirements from subject matter experts.

In this example, the LLM did a nice job of identifying gaps and recommending improvements, saving considerable time and rework.

3

Submit your answers to the LLM for consideration.



Reviewing the Requirements (cont')

Expand the context of the LLM through user journeys.

Coach's Notes



Next, propose the primary user journeys we should model based on these requirements.



Based on the refined requirements, we should model our API design around the primary user journeys. These journeys represent the key goals a user or system will try to accomplish. Proposing these helps ensure our API endpoints are logical, intuitive, and map directly to real-world use cases.

Here are the five primary user journeys we should model:

1. Guest Customer: Find and Purchase a Book

This is the fundamental "happy path" for a new customer, focusing on discovery and a frictionless checkout experience.

- **Goal:** A new user finds a book and buys it without creating an account.
- **Journey Steps:**
 1. Discover: The user searches for books using filters (e.g., genre, format).
 2. Evaluate: The user views the details of a specific book (description, price, reviews).
 3. Select: The user adds the book to their shopping cart.
 4. Checkout: The user initiates checkout, providing their email, shipping address, and credit card information.
 5. Confirm: The user reviews the final order total, including the flat-rate shipping, and places the order.
 6. Receive ID: The user sees a confirmation page displaying their new Order ID.

2. Registered Customer: Account Creation and Management

.... truncated for space...

3. Registered Customer: Purchase with Order Management

.... truncated for space...

4

It can be helpful to ask the LLM to recommend some user journeys to accelerate the API design effort.

5

In this example, the LLM recommended a total of 5 user journeys for consideration.

Blend these with your own to ensure you consider happy and error paths. The steps in your user journeys will help to drive your API design, so spend sufficient time at this step.

6

Ask the LLM to then review everything one final time.



Review the requirements and identify any recommendations that should be considered before proceeding



Reviewing the Requirements (cont')

Identify remaining gaps and open questions.



Based on the consolidated business profile, here are five key recommendations to consider before proceeding with the API design. These suggestions aim to improve customer experience, future-proof the system, and clarify potential ambiguities.

1. Offer Flexible Shipping for Mixed Carts

The current requirement states that an order with both in-stock and pre-order items will be held until all items are ready to ship.

- **Potential Issue:** This can create a poor customer experience, as a customer might wait months for an in-stock book, leading to confusion and support requests.
- **Recommendation:** We should empower the customer by providing a choice during checkout. The API should support an option to either:

1. **Ship Everything Together:** The current default behavior.

2. **Split the Shipment:** Ship in-stock items now and pre-order items later. This would create two separate shipments (and potentially incur a second shipping fee).

This approach provides flexibility and manages customer expectations upfront.

2. Define a Clear Product Hierarchy

The requirements list different book formats (hardcover, paperback, ebook) but don't specify how they relate to each other.

- **Potential Issue:** Treating each format as a completely separate, unrelated product can lead to duplicated data (e.g., the same synopsis and author info repeated for each format) and makes it difficult for a user to discover all available versions of a single book.
- **Recommendation:** Model the catalog with a parent "Work" entity that groups multiple "Product" variations.
 - o A **Work** would represent the book itself (title, author, genre, reviews).
 - o Each **Product** linked to that Work would represent a specific format (e.g., the hardcover version with its own ISBN, price, and inventory).

.... truncated for space...

7

Coach's Notes

For well-known domains, the LLM will likely identify some gaps or recommendations.

Notice that in this example, the LLM suggests different ways to model the domain, speeding up both understanding and design efforts.

While not all of these will apply, it may help you incorporate some future-proofing API design elements.

Finally, incorporate any recommendations, as desired, and request a fresh set of requirements will all changes incorporated.



Integrate items 1,4, and 5 to the requirements and update the consolidated business profile. Then provide a summary of the full set of requirements for review.



Reviewing the Requirements (cont')

Reviewing the Requirements



JSON's Bookstore: Updated Consolidated Business Profile

1. Business Overview

- **Name:** JSON's Bookstore
- **Business Model:** An online e-commerce retailer specializing exclusively in the sale of physical and digital books.
- **Target Consumers:** The platform serves individual online shoppers through a dedicated web interface and mobile app, as well as business partners who integrate with the catalog for informational purposes.

2. Customer Profiles & Features

The platform supports two distinct types of customers:

- **Guest Customers:**
 - o Can browse, search, and filter the entire book catalog.
 - o Can add items to a cart and complete a purchase by providing their information at checkout.
 - o Do not have access to a persistent profile or order history.
- **Registered Customers:**
 - o Possess a secure account with login credentials.
 - o Can view a complete history of their past and pending orders.
 - o Can manage their profile by saving multiple shipping addresses and payment methods for a faster checkout experience.
 - o Have the ability to cancel an order while it is in the RECEIVED status, before it moves to fulfillment.

3. Product Catalog & Offerings

- **Core Products:** Books offered in three primary formats: Hardcover, Paperback, and Ebook.
- **Product Attributes:** Each book is defined by a rich set of data, including Title, Author, ISBN, Genre, Format, Price, Language, Previews, Customer Reviews, and Ratings.
- **Curated Bundles:** The store offers pre-packaged collections of books (e.g., "Staff Picks," "Publisher Specials") that are sold as a single, unique product.
- **Product Discovery:** Customers can find books by:
 - o Searching on specific details (Title, Author).
 - o Filtering results by criteria like Price Range and Format.
 - o Browsing curated lists and categories.

8

Coach's Notes

Now it is time to review everything, compose any prompts needed to improve the requirements, and then move into the ADDR process.

Notice in this example that the requirements are far more complete and clarify a number of issues compared to the original set of basic requirements supplied.

This offers a huge lift for product leaders as well as API designers, to ensure that the problem space is better understood before proceeding into the ADDR process.



Align

Starting the Align Phase

Transform clarified requirements into job stories and activity steps

Coach's Notes

Human Prompt Engineer

Ok, let's proceed to the Align phase of ADDR. Remember, we are API architects that need to translate requirements into an API design. Here are the rules and restrictions for this effort:

- * Until I tell you otherwise, let's stay in the Align phase and not blend any activities from the other phases of ADDR
 - * By staying only in the Align phase, we will need to compose a set of unifying job stories and associated activities and activity steps
 - * Job stories will use the standard format of WHEN, I WANT TO, SO I CAN
 - * Job stories should be unifying job stories, meaning that they have a clear and broad scope sufficient to result in an outcome rather than completing a task
 - * Job stories must be numbers JS1..n for easy reference between us and between artifacts
 - * Activity steps are actions that the user will perform via the API. Do not include any UI-specific actions by the user, such as scrolling, reading, paging through results.
- Focus on action-specific steps
- * The activity steps will be grouped into activities, which map to capabilities
 - * We will not consider the Align phase complete until we have both artifacts completed
 - * We will use business language that benefits the API consumer rather than any internal or operational terms
 - * We will not focus on anything related to HTTP yet
 - * When we have determined that we have completed the Align phase, you will package the final Align deliverable as a single, clean reference sheet so it's ready for hand off to the Define phase

Let's start by having you explain the phase, these artifacts, and why it is important so that we are clear.

9

When beginning an ADDR coaching session, it is important to restrict the model from moving outside of the phase. Failing to do so will result in the model attempting to design the API too soon, missing important steps and key details.

The initial prompt also asks for an explanation of the key artifacts within the Align phase.

This will help those not familiar with ADDR and serves to ensure the model has full understanding of the process.



Align

Starting the Align Phase (con't)

Transform clarified requirements into job stories and activity steps

Coach's Notes



LLM Response

The Align phase is the crucial bridge between the business requirements we've just defined and the technical API design that will follow. Its entire purpose is to ensure that everyone involved—architects, product managers, and developers—shares a common understanding of what the API needs to do from the perspective of the consumer before we start designing how it will do it. It forces us to think "outside-in."

Unifying Job Stories

A Unifying Job Story is a simple but powerful tool that shifts our focus from features to outcomes. It captures the user's motivation and desired goal, not just a single task. Format: We'll use the standard WHEN..., I WANT TO..., SO I CAN... structure.

- o WHEN: This describes the context or situation that triggers the need.

- o I WANT TO: This is the core action or goal the user wants to accomplish.

- o SO I CAN: This describes the user's ultimate motivation or desired outcome.

- "Unifying" Scope: The key is that these stories are broad and outcome-driven. For example, a bad, task-focused story would be "When I'm on the product page, I want to press the add to cart button, so I can put an item in my cart." A good, unifying story is "When I've decided on the books I want, I want to complete the purchase and confirm my order, so I can receive my books soon." The second story covers the entire checkout outcome.

Activities and Activity Steps

This artifact breaks down the "how" behind the job stories into concrete, API-level actions.

- **Activities:** These are high-level capabilities that group related actions. Think of them as chapters in the API's story, like "Manage Shopping Cart" or "Process an Order."

- **Activity Steps:** These are the specific, discrete actions a user performs through the API to complete an activity. This is the most critical part for our design work.

- o **API-Focused:** As you said, these steps must only be actions the API enables. We will completely ignore UI-specific interactions. For example, instead of "click the 'Apply Coupon' button," the activity step would be "Apply a discount code to the cart."

10

The response from the LLM shows its understanding of ADDR. It focuses on two sets of artifacts: job stories and activity steps.

Job stories help to capture details about each user journey, including the desired outcome to be achieved. This will shape our acceptance tests and help us focus the scope of the effort throughout the design process. Job stories also support the delivery process.



Align

Generating Job Stories

Produce a starting set of job stories based on the clarified requirements.

Coach's Notes



Excellent. Using the requirements above, let's write one or more unifying job stories for each of the major flows. Remember that job stories capture jobs to be done rather than steps in a process, so let's keep them high-level. Be sure to number the job stories using the format JS1..n, so that we can refer to them easily as we work through the ADDR process



Excellent. Let's create the Unifying Job Stories. These will serve as the foundation for our API's capabilities, ensuring we're focused on the outcomes our consumers want to achieve. Here are the unifying job stories for the major flows we've identified.

JS1: Complete a One-Time Purchase

-
- **WHEN** I find the book(s) I want to read,
- **I WANT TO** provide my shipping and payment information for a one-time purchase,
- **SO I CAN** confirm my order and receive my books without the friction of creating a permanent account.

JS2: Manage a Persistent Customer Profile

- **WHEN** I anticipate becoming a repeat customer,
- **I WANT TO** create and manage a secure account with my addresses and payment methods,
- **SO I CAN** streamline future checkouts and have a consolidated view of all my orders.

JS3: Discover Relevant Books

- **WHEN** I'm looking for a new book,
- **I WANT TO** easily search, filter, and browse the store's catalog and view detailed product information,
- **SO I CAN** discover a title that matches my interests and make a confident purchasing decision.

JS4: Manage an Order After Purchase

- **WHEN** my plans change after making a purchase,
- **I WANT TO** review my pending orders and cancel one before it ships,
- **SO I CAN** manage my purchases flexibly and avoid an unwanted shipment.

JS5: Synchronize Product Catalog Data

- **WHEN** we need to display JSON's Bookstore products on our partner platform,
- **I WANT TO** programmatically access their catalog and availability information,
- **SO I CAN** ensure our listings are always accurate and up-to-date.

11

The first step is to ask the model to generate a set of unifying job stories. These are the “big picture” job stories that drive acceptance tests, design, and documentation.

12

Some models may get a bit too detailed with the **SO I CAN** portion of a job story. Be sure to check the output or ask the model to double check itself to ensure that your job stories result in a desired outcome.

In some cases, generated job stories might be better captured as workflow activities or activity steps.



Align

Improving the Job Stories

Unify, split, or modify the generated job stories.



Looks good. Let's renumber them. Change JS3 to JS1 and the current JS1 should be JS2. The current JS2 should be JS3



Understood. Here is the reordered list of Unifying Job Stories with the new numbering.

JS1: Discover Relevant Books

- **WHEN** I'm looking for a new book,
- **I WANT TO** easily search, filter, and browse the store's catalog and view detailed product information,
- **SO I CAN** discover a title that matches my interests and make a confident purchasing decision.

JS2: Complete a One-Time Purchase

-
- **WHEN** I find the book(s) I want to read,
- **I WANT TO** provide my shipping and payment information for a one-time purchase,
- **SO I CAN** confirm my order and receive my books without the friction of creating a permanent account.

JS3: Manage an Order After Purchase

- **WHEN** my plans change after making a purchase,
- **I WANT TO** review my pending orders and cancel one before it ships,
- **SO I CAN** manage my purchases flexibly and avoid an unwanted shipment.

JS4: Manage a Persistent Customer Profile

- **WHEN** I anticipate becoming a repeat customer,
- **I WANT TO** create and manage a secure account with my addresses and payment methods,
- **SO I CAN** streamline future checkouts and have a consolidated view of all my orders.

JS5: Synchronize Product Catalog Data

- **WHEN** we need to display JSON's Bookstore products on our partner platform,
- **I WANT TO** programmatically access their catalog and availability information,
- **SO I CAN** ensure our listings are always accurate and up-to-date.

Coach's Notes

13

After the job stories were generated from our requirements, we found that the flow was out of order, so we renumbered the job stories to better reflect the natural progression of the user through the process.

You may also find that some job stories may need to be removed or rephrased.

14

Review the changes to ensure that you have captured job stories that reflect outcomes, not just steps in a workflow.

When ready, we will generate the final artifact of the Align phase: activities and steps.



Align

Expanding Jobs Into Activity Steps

Activity steps help to identify the steps required to complete each job story.



Let's break each unifying job story down into the activities and activity steps that are required to go from the situation to the expected outcome. Avoid including steps taken in a UI such as scrolling or entering criteria, instead focusing on the action of submitting the criteria. Remember that activities represent milestones or incremental groups of work, while the activity steps represent a single action taken.



Of course. Let's break down each unifying job story into its constituent Activities and Activity Steps.

JS1: Discover Relevant Books

This job is about finding and evaluating books in the catalog.

- **Activity: Discover Books in the Catalog**
 - o Search for books based on criteria (e.g., title, author).
 - o Browse a list of books by category.
 - o View a curated list of books (e.g., best-sellers).
- **Activity: Evaluate a Specific Book**
 - o Retrieve detailed information for a book.
 - o View reviews and ratings for a book.

JS2: Complete a One-Time Purchase

This job covers the entire guest checkout process from cart management to order confirmation.

- **Activity: Manage Shopping Cart**
 - o Add a book or bundle to the cart.
 - o View the contents of the cart.
 - o Update the quantity of an item in the cart.
 - o Remove an item from the cart.
 - o Apply a discount code to the cart.
- **Activity: Provide Checkout Information**
 - o Submit guest customer information (e.g., email).
 - o Submit a shipping address.
 - o Get available shipping options for the cart.
 - o Select a shipping option.
 - o Submit payment information.
- **Activity: Finalize the Order**
 - o Create an order from the cart.
 - o Retrieve the order confirmation details.

.... truncated for space...

Coach's Notes

15

With an initial set of job stories established, the next step is to break down the job stories into the activities and steps required to perform the job/task.

The requirements and user journey details we clarified at the start will help to drive this effort.

16

Some models stumble on this step, as they want to mimic the user clicking, scrolling, and entering data actions before submitting a request.

This prompt asks the LLM to focus the model on action steps only.



Align

LLM-Generated Job Stories with Activities

Example output generated by LLM.

Job Story ID	Activity	Activity Step	Description
JS1	Discover Books in the Catalog	Search for books based on criteria	Submits a query to find books matching details like title or author.
JS1	Discover Books in the Catalog	Browse a list of books by category	Fetches a collection of books belonging to a specific genre.
JS1	Discover Books in the Catalog	View a curated list of books	Retrieves a special collection, such as best-sellers or new releases.
JS1	Evaluate a Specific Book	Retrieve detailed information for a book	Gets all public data for one book, including synopsis and price.
JS1	Evaluate a Specific Book	View reviews and ratings for a book	Fetches customer-submitted reviews and ratings for a book.
JS1	Evaluate a Specific Book	Get recommendations related to a book	Retrieves a list of books <u>similar to</u> the selected one.
JS2	Manage Shopping Cart	Create a new shopping cart for the session	Initializes a new, empty cart and associates it with the user's session.
JS2	Manage Shopping Cart	Add a book or bundle to the cart	Adds a specific product and quantity to the user's current cart.
JS2	Manage Shopping Cart	View the contents of the cart	Retrieves all items currently in the user's shopping cart.
JS2	Manage Shopping Cart	Update the quantity of an item	Changes the number of units for a specific item in the cart.
JS2	Manage Shopping Cart	Remove an item from the cart	Deletes a specific item from the shopping cart.
JS2	Manage Shopping Cart	Apply a discount code to the cart	Submits a promotional code to be validated and applied to the cart.
JS2, JS3	Provide Checkout Information	Submit guest customer information	Provides non-persistent user details, like an email, for the order.
JS2, JS3	Provide Checkout Information	Submit a shipping address	Provides the destination address for the order's shipment.
JS2, JS3	Provide Checkout Information	List the customer's saved shipping addresses	Retrieves saved addresses for a logged-in user to select from.

Job Story ID	Activity	Activity Step	Description
JS2, JS3	Provide Checkout Information	Get available shipping options	Retrieves shipping methods and costs based on the cart contents.
JS2, JS3	Provide Checkout Information	Select a shipping option	Confirms the chosen shipping method for the order.
JS2, JS3	Provide Checkout Information	Submit payment information	Provides the necessary payment details to authorize the transaction.
JS2, JS3	Provide Checkout Information	List the customer's saved payment methods	Retrieves saved payment methods for a logged-in user to select from.
JS2	Finalize the Order	Create an order from the cart	Submits the finalized cart to create a persistent order record.
JS2	Finalize the Order	Retrieve the order confirmation	Fetches the summary details of the successfully created order.
JS3	Manage Customer Account	Create a new customer account	Registers a new user with the provided credentials.
JS3	Manage Customer Account	Authenticate a customer	Verifies credentials to create an active user session (sign in).
JS3	Manage Customer Account	Invalidate a customer session	Ends the current user session (sign out).
JS3	Manage Customer Account	Retrieve the customer's profile	Fetches the account information for the authenticated user.
JS3	Manage Customer Account	Update the customer's profile	Modifies the account information for the authenticated user.
JS3	Manage Saved Addresses	Add a new shipping address	Saves a new address to the customer's profile.
JS3	Manage Saved Addresses	List all saved shipping addresses	Retrieves all addresses associated with the customer's profile.
JS3	Manage Saved Addresses	Update a saved shipping address	Modifies an existing address in the customer's profile.
JS3	Manage Saved Addresses	Remove a saved shipping address	Deletes an address from the customer's profile.

Example of a generated set of job stories and activity steps that will inform our API design



Align

Checking the Align Phase

Run a gap analysis to ensure we didn't miss anything.



Review the job stories and activity steps, looking for gaps in the artifacts due to missing or unclear requirements. Present anything you find as a numbered item to review for any updates



Here is a review of the artifacts, highlighting three potential gaps for clarification before we finalize the Align phase.

1. Missing Step for Cart-to-User Association

Finding: In JS2 (Complete a One-Time Purchase), the Manage Shopping Cart activity has steps to add, view, and update a cart. However, there is no explicit step to create or associate the cart with the user's session. A guest user needs a persistent cart that exists before they add the first item and remains associated with them throughout their visit. Recommendation: Consider adding an activity step like "Create a new shopping cart for the session" to the Manage Shopping Cart activity to represent this initial action.

2. Ambiguity in Handling Saved Information at Checkout

Finding: JS3 (Manage a Persistent Customer Profile) defines how a user saves addresses and payment methods. The associated job story mentions streamlining future checkouts, but there is no explicit activity step in any job story that describes using this saved information. Recommendation: We should add activity steps to the Provide Checkout Information activity for a registered user's journey. These could include "List the customer's saved shipping addresses" and "List the customer's saved payment methods" to make it clear how the API facilitates a faster checkout.

3. No Activity for Price or Promotion Details

Finding: The Manage Shopping Cart activity includes a step to "Apply a discount code to the cart," and the requirements mention bundles. However, there is no activity step that allows a user to understand the price breakdown, see the result of the discount, or view the details of a bundle before adding it to the cart. A customer needs to know the price to make a purchasing decision. Recommendation: Add a step to the Evaluate a Specific Book activity in JS1 called "Retrieve pricing and promotion details for a book or bundle." This clarifies how a customer gets price information, which is fundamental to the discovery and evaluation process.



s

Coach's Notes

17

LLMs are great at reconciling sets of data. Leverage this capability throughout the ADDR process to ensure that you didn't miss anything along the way.

18

In this example, the LLM found three gaps outside of the original requirements clarification effort.

Review the recommendations and either ignore or incorporate them into your work. In some cases, you may need to gather feedback from your experts before proceeding. Better now than when the cost of change is high.



Align

Checking the Align Phase (con't)

Run a coverage check to ensure we didn't miss anything.



Run a coverage check and produce a coverage matrix so we can be confident the Align phase is airtight before moving forward. Then, review the job stories and activity steps, looking for gaps in the artifacts due to missing or unclear requirements. Present anything you find as a numbered item to review for any updates

18



Of course. Performing a coverage check is the perfect way to validate our work in the Align phase.

A coverage check ensures that every business requirement we defined is covered by at least one activity step. This process confirms that our API design will be complete and prevents critical gaps from being discovered late in the process.

Below is the coverage matrix mapping our key business requirements to the corresponding Job Story IDs and Activities.

API Coverage Matrix

Business Requirement	Job Story ID	Activity	Status
Customer can browse and search the catalog	JS1	Discover Books in the Catalog	✓ Covered
Customer can view detailed book information	JS1	Evaluate a Specific Book	✓ Covered
.... truncated for space...			

Analysis

The coverage matrix shows that all business requirements are fully covered by the job stories and activities we have defined. Every function the business needs is mapped to a clear, consumer-focused action that the API will enable.

With this confirmation, we can confidently conclude that the Align phase is complete and we are ready to proceed to the Define phase.

Coach's Notes

(con't)

Once we have reconciled everything, perform a final cross-check and address any remaining issues.

Then, proceed to the Define phase, where we will start to map the job stories and activity steps into one or more API profiles.



Define

Starting the Define Phase

Turn the Align artifacts into one or more API profiles that reflect scope and purpose.



Define

Coach's Notes

Human Prompt Engineer

The Align phase is now complete. Let's start the Define phase. Here are the rules and restrictions for this effort:

- * Until I tell you otherwise, let's stay in the Define phase and not blend any activities from the other phases of ADDR
- * By staying only in the Define phase, we will need to identify the API boundaries and compose an API profile for each API that we have identified
- * We will not consider the Define phase complete until we have identified the boundaries and composed API profile(s), API resource(s), published Event(s) for each API operation, identified the API resource relationships, and a sequence diagram or sequence example of how the API would be used to complete each job story
- * API boundaries will be summarized in a table that contains the following columns: Boundary Name, Boundary Description, Job Story(s)
- * API profiles will be summarized in a table that contains the following columns: Operation Name, Description, Participants, Resource(s), Emitted Events, Operation Characteristics, Operation Details
 - * The Operation Name will be in the format of operationName()
 - * The Participants include one or more roles or personas that will need access to the operation
 - * The Resource(s) include the primary and possible secondary resources involved in the operation
 - * Emitted Events includes one or two events that the API will publish when the operation completes successfully, allowing for future
 - * Operation Characteristics identifies the safe, unsafe, or idempotent design of the operation, and if the operation should be designed as synchronous (default) or asynchronous
 - * Operation Details include the request and response values, such as identifiers, resource representations, and filters. Keep in mind that we don't want to design the HTTP details yet, so let's keep this to basic details
- * API resources will include a business-friendly name and a list of properties. Keep in mind that we aren't ready to define the schema details yet, so don't include any JSON schema details just model the resource independent of HTTP or design details
- * Emitted Events should include at least one event in most cases using the format of ResourceName.EventName. For example, Project.Created
- * We will use business language that benefits the API consumer rather than any internal or operational terms
- * We will not focus on anything related to HTTP yet
- * We will identify boundaries that benefit the API consumer rather than the API producer, to ensure a better developer experience (DX) and cohesive API profiles
- * When we have determined that we have completed the Define phase, you will package the final Define deliverable as a single, clean reference sheet so it's ready for hand-off to the Design phase

Let's start by having you explain the phase, these artifacts, and why it is important so that we are clear.

19

When your job stories and activity steps are clear, proceed to the Define phase. This is the phase where we start to sketch out what our API will need to do. We will also look for any boundaries where we may want to separate APIs.

For the prompt, we will tell the LLM to stay within the Define phase to avoid jumping too far ahead. If necessary, you can always go back and adjust something from the previous phase.

20

As before, we ask the model to move to the Define phase and explain the phase to us.



Define

Starting the Define Phase (con't)

Turn the Align artifacts into one or more API profiles that reflect scope and purpose.



Define

Coach's Notes



LLM Response

Excellent. Let's proceed to the Define phase. The goal of this phase is to translate the user-focused "what" we captured in the Align phase (our job stories and activities) into a formal, structured model of the API's surface. We're creating a detailed blueprint that describes the API's capabilities, the data it handles, and its behavior, all while remaining independent of any specific protocol like HTTP.

Key Artifacts for the Define Phase

We'll build out our API's blueprint by creating a series of connected artifacts.

1. API Boundaries

First, we'll identify the API Boundaries. Think of these as logical, consumer-focused groupings of related capabilities. Instead of exposing one massive, monolithic API, we define distinct "products" or "domains" that are cohesive and easier for developers to understand and use. This is a critical step for creating a great developer experience (DX). We will summarize these in a table with the columns: Boundary Name, Boundary Description, and the Job Story(s) it serves.

2. API Profiles

For each boundary, we will compose one or more API Profiles. A profile is a detailed, protocol-agnostic contract that describes every operation the API offers. This is the core artifact of the Define phase. Each operation in the profile will be detailed using the following columns:

- **Operation Name:** The action, named like a function (e.g., `getBookDetails()`).
- **Description:** A clear explanation of what the operation does.
- **Participants:** The user roles who will use this operation (e.g., Customer, Partner).
- **Resource(s):** The primary data entities involved.
- **Emitted Events:** Notifications published upon successful completion (e.g., `Order.Created`).
- **Operation Characteristics:**
 - Safety: Safe (read-only), Unsafe (modifies data), or Idempotent (can be repeated without changing the outcome).
 - Execution: Synchronous (immediate response) or Asynchronous (response later).
- **Operation Details:** A high-level view of the request inputs (like identifiers or filters) and the response outputs.

3. API Resources and Events

To support the profiles, we'll also formally define our resources and events.

- **API Resources:** These are the business objects or "nouns" of our API (e.g., Book, Order, Customer). We'll give each a business-friendly name and list its properties, without getting into technical schema details like data types.
- **Emitted Events:** These are the signals our API sends out to notify other systems of important occurrences. Defining these from the start allows us to build a more reactive and extensible architecture.

By the end of this phase, we will have a complete, protocol-agnostic model of our API's structure and behavior, ready to be translated into a specific protocol (like REST) in the final Design phase. Let's begin by identifying the API boundaries. We will use the previous job stories and activity steps from this session as source material for the Define phase. Do not hypothesize regarding any necessary details regarding job stories and activity steps.

21

The model will also outline the artifacts that we want to produce to ensure that our API's scope and responsibilities are clear.

As you may recall, the Define phase strives to identify the API operations and characteristics that are important to your API, before you begin to apply an API design style.

This step helps us to make decisions later regarding HTTP methods, response codes, and other elements that will ensure your API is able to evolve over time.



Define

Identifying Candidate Boundaries

Request the LLM to evaluate potential boundaries, adjusting as needed with prompts.

Coach's Notes



Recommend any candidate boundaries as a first version of the boundaries table. Then, provide a summary of why the boundaries work and any additional considerations, including overlap or boundaries that could be unnecessary based on requirements provided.

Be sure to suggest boundaries that align with a better developer experience rather than strictly aligning boundaries to domain concepts. Your goal is to determine where one API or service boundary should end and another should begin, based on the business domain, invariants, consistency needs, and language differences within the domain.

Rules for Identifying API Boundaries

1. Do *not* distribute a single domain model across APIs

Look for areas where splitting the model would require synchronous remote calls. If so, keep them together.

2. Each API corresponds to a single Bounded Context

A boundary is valid only if it represents:

- * a distinct domain language
- * a cohesive responsibility
- * its own invariants
- * its own data ownership
- * independent lifecycle and scaling needs

3. Invariants must be enforced *within* a boundary, not across them

If enforcing a rule requires strong consistency between two concepts, they belong together.

4. Avoid synchronous cross-boundary calls during domain operations

Cross-boundary communication must *not* be needed for:

- * validations
 - * invariant checks
 - * business rules
 - * transaction completion
- If it is needed, merge the boundaries.

.... truncated for space...

22

Some models are better than others at finding API boundaries. Models trained on smaller, microservice-based architectures tend to recommend smaller boundaries.

Note that this prompt is quite large, so it has been truncated in this report. Refer to the Resources page at the end of the report for a link to the full prompt.

Remember: Boundaries should be for the benefit of the consumer, not for optimizing your delivery process.



Define

Identifying Candidate Boundaries

Request the LLM to evaluate potential boundaries, adjusting as needed with prompts.



Based on the job stories and activities from the Align phase, here are the recommended API boundaries.

Boundary Name	Boundary Description	Job Story(s)
Catalog API	Provides all capabilities for discovering and evaluating books and bundles available for sale.	JS1, JS5
Orders API	Manages the entire lifecycle of a customer's purchase, from cart creation through checkout and post-purchase actions.	JS2, JS4
Customers API	Handles the creation and management of persistent customer profiles, including saved addresses and payment methods.	JS3

These three boundaries provide a clean separation of concerns based on the primary consumer goals.

Summary and Considerations

These three boundaries provide a clean separation of concerns based on the primary consumer goals.

• Why They Work:

- o **Catalog API:** This creates a single, authoritative source for all product-related information. It serves both public consumers browsing the store (JS1) and partner systems needing to synchronize data (JS5). Grouping these read-heavy discovery operations creates a cohesive and highly cacheable API surface.
- o **Orders API:** This boundary logically groups the entire transactional workflow. The "shopping cart" is conceptually the first step of an "order," so placing cart management and order finalization together makes sense. Including postpurchase actions like cancellations (JS4) in the same boundary provides a complete view of the order lifecycle.
- o **Customers API:** This boundary is focused entirely on the identity and data of a registered user (JS3). Separating customer profile management from the ordering process allows for a clear security and privacy model. For instance, this API would require stricter authentication than the public-facing Catalog API.

• Additional Considerations:

- o **Overlap:** There is a necessary and intentional overlap. The Orders API will interact with data managed by the Catalog API (e.g., adding a book to a cart) and the Customers API (e.g., associating an order with a registered customer). However, the capabilities remain distinct. The Orders API uses a book's ID; it does not manage the book's details.
- o **Necessity:** All three boundaries are necessary as they map directly to distinct, high-level jobs to be done. We could have combined Orders and Customers, but separating them creates a better, more secure design that aligns with modern microservices principles. There do not appear to be any unnecessary boundaries at this stage.

23

Coach's Notes

In this example, the boundaries match what is typically found for an order flow.

The Catalog API supports browsing existing inventory.

The Orders API allows the user to manage their cart and place their order.

The Customers API supports account creation and management.

Be prepared to rework your boundaries as you go, especially after evaluating the sequence diagrams that will be produced later in this phase.



Define

Identifying Candidate Boundaries

Request the LLM to evaluate potential boundaries, adjusting as needed with prompts.

Coach's Notes



Let's proceed to identifying the API resources we will need by reviewing the job stories and activity steps



Based on the job stories and activity steps, here are the primary API resources we will need.

Catalog API Resources

These resources are the core entities related to product discovery and evaluation.

- **Book**
 - o Description: Represents a single, purchasable version of a book (e.g., the hardcover edition).
 - o Properties: bookId, workId, title, author, isbn, format, price, language, synopsis, coverImage.
- **Work**
 - o Description: Represents the abstract concept of a book, grouping all its different formats.
 - o Properties: workId, title, author, genre, averageRating.
- **Bundle**
 - o Description: A curated collection of books sold as a single unit.
 - o Properties: bundleId, name, description, price, containedBooks.
- **Review**
 - o Description: A customer-submitted review and rating for a specific Work.
 - o Properties: reviewId, workId, customerName, rating, comment.
- **Inventory**
 - o Description: The abstracted availability status for a purchasable item.
 - o Properties: itemId (can be a bookId or bundleId), status (e.g., IN_STOCK, OUT_OF_STOCK)

Orders API Resources

These resources are involved in the entire transactional process, from shopping to post-purchase.

- **Cart**
 - o Description: A temporary container for items a customer intends to purchase.
 - o Properties: cartId, customerId (if applicable), items, subtotal, discounts, total.
- **Order**
 - o Description: A confirmed and placed purchase record.
 - o Properties: orderId, customerId (if applicable), items, shippingAddress, shippingMethod, total, status, placedDate.

.... truncated for space...

24

Once you have a preliminary set of boundaries, the next step is to ask the model to help us craft our API resources that our operations will manage.

25

LLM models have a wide range of domain understanding, allowing you to speed up this process considerably.

If you need to leverage some specific industry standards, mention this here as well.

Spend as much time here as needed to ensure that your API design reflects your domain models and relationships.



Define

Generating API Profiles

Allow the LLM to propose an API name, description, and recommended operations.

Coach's Notes



Let's generate an API profile for each boundary, proposing a business-friendly name and description for each API that denotes its scope and purpose along with the API operations mapped from the related activity steps. For the characteristics column value of each operation, use "safe" if read-only, "unsafe" if the operation isn't idempotent, or "idempotent" if the operation should be idempotent. Also include "sync" or "async" based on whether the operation will return results or need to operate in the background. Finally add "bulk import" only if the operation is performing a bulk import of data

26

With our boundaries and resources established, ask the model to generate a first-pass API profile that contains all API operations, the resources that will be referenced, and the operational characteristics.



Here are the proposed API profiles for each boundary, with operations mapped directly from our activity steps.

Catalog API Profile

API Name: Product Catalog

API Description: Provides public access to discover, browse, and evaluate all books and bundles sold in JSON's Bookstore. It is the single source of truth for product information for both customers and external partners.

Operation Name	Description	Participants	Resource(s)	Emitted Events	Characteristics	Operation Details
searchBooks()	Finds books matching specific query criteria.	Customer, Partner	Book	(none)	safe, sync	Request: searchQuery, filters Response: listOf(Book)
browseByCategory()	Retrieves a list of books within a given category.	Customer, Partner	Book	(none)	safe, sync	Request: categoryId Response: listOf(Book)
getCuratedList()	Retrieves a specially curated list of books.	Customer, Partner	Book, Bundle	(none)	safe, sync	Request: listId Response: listOf(Book or Bundle)
getBookDetails()	Retrieves all detailed information for a single book.	Customer, Partner	Book, Work	(none)	safe, sync	Request: bookId Response: Book, Work
getBookReviews()	Fetches all customer reviews for a book's work.	Customer, Partner	Review, Work	(none)	safe, sync	Request: workId Response: listOf(Review)
getRecommendations()	Retrieves a list of recommended books.	Customer, Partner	Book	(none)	safe, sync	Request: bookId Response: listOf(Book)
getInventoryStatus()	Gets the availability status for a product.	Partner	Inventory	(none)	safe, sync	Request: itemId Response: Inventory

.... truncated for space...

27

Remember that we aren't looking to add HTTP details yet. So, no POST, GET, PUT, or URL paths are needed for this step.

Review your API profiles to see if everything looks as expected.

Use prompts to make any adjustments as needed to your API profile, especially after the diagram in the next steps.



Define

Generating Flow Diagrams

Produce flow diagrams of the API resolving each job story using Mermaid.js format.



Generate the sequence diagrams for each job story using Mermaid format



Here are the sequence diagrams for each job story, rendered in Mermaid format. This diagram shows a consumer first searching the catalog and then retrieving the specific details and reviews for a book they are interested in.

JS1: Discover Relevant Books

This diagram shows a consumer first searching the catalog and then retrieving the specific details and reviews for a book they are interested in.

```
sequenceDiagram
    participant Consumer
    participant Catalog API
    Consumer->>Catalog API: searchBooks(query="API Design")
    activate Catalog API
    Catalog API-->>Consumer: listOf(Book)
    deactivate Catalog API
    Consumer->>Catalog API: getBookDetails(bookId="bk_123")
    activate Catalog API
    Catalog API-->>Consumer: Book representation
    deactivate Catalog API
    Consumer->>Catalog API: getBookReviews(workId="wk_abc")
    activate Catalog API
    Catalog API-->>Consumer: listOf(Review)
    deactivate Catalog API
```

JS2: Complete a One-Time Purchase

This diagram illustrates the guest checkout flow, from creating a cart to placing the final order.

.... truncated for space...

28

You may notice that your API profile looks complete at-a-glance. However, it isn't complete until it can realize the outcomes of your job stories.

The next step is to ask the model to generate sequence flow diagrams to demonstrate how your API will take a user from their problem (WHEN) to their desired outcome (SO I CAN) for each job story.

29

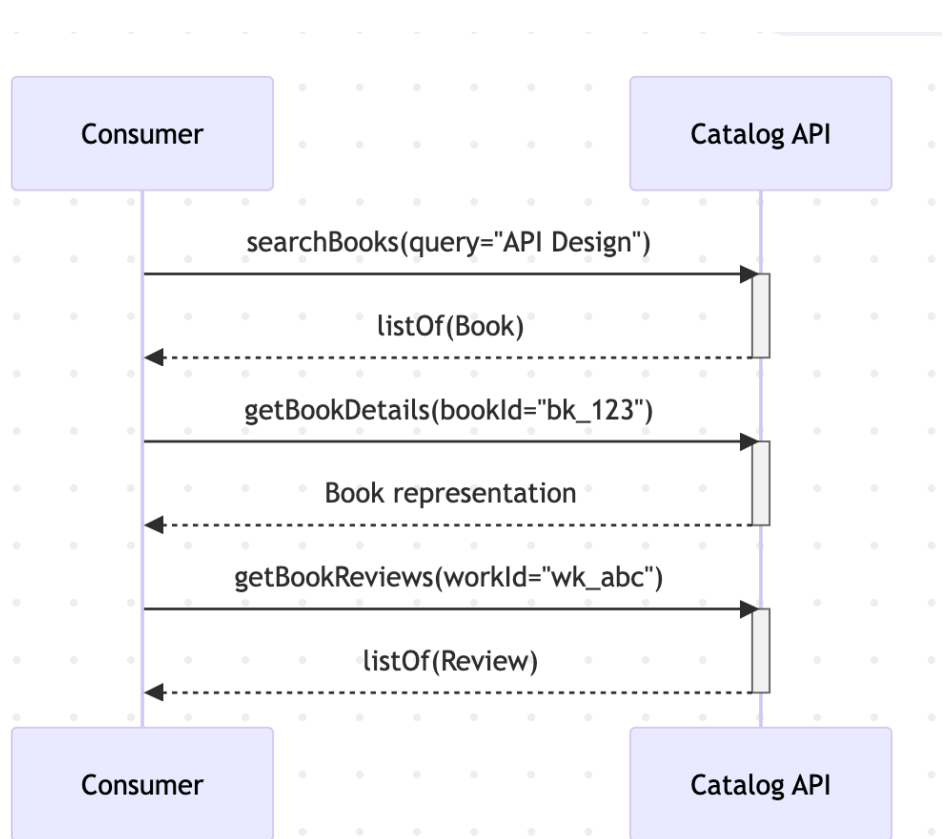
Manually, this can be a time-consuming process, but the LLM makes it easier to quickly generate diagrams using previous artifacts, then review them for gaps.



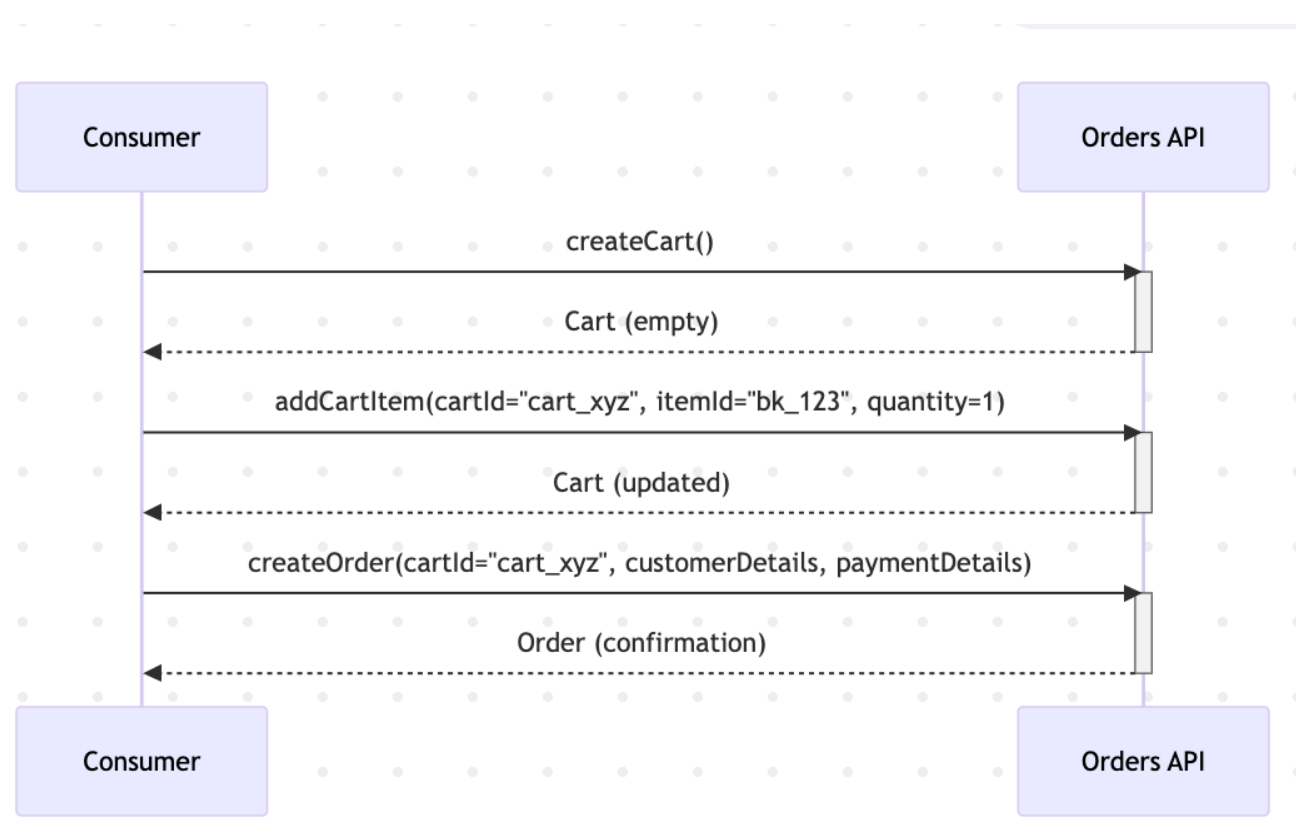
Define

LLM-Generated Artifact: Sequence Diagrams

Example output generated by LLM.



Example 1: JS1 Sequence Diagram



Example 2: JS2 Sequence Diagram



Define

Checking the Define Phase

Ask the LLM to review all assets and identify any gaps with recommended fixes.

Coach's Notes



Look for missing operations in the flow that haven't been considered. Recommend improvements to the API profiles



Based on a review of the flows, here are two recommendations for new operations to improve the API profiles.

Recommendations for New Operations

1. Add an Operation to Retrieve a Customer's Cart

Finding: The sequence diagrams assume a user always knows their cartId. However, a returning registered customer who added items to their cart in a previous session and then logged out would have no way to retrieve their persistent cart. The current `getCart(cartId)` operation in the Orders API requires the ID, which the user would not have.

Recommendation: Add a `getCustomerCart()` operation to the Orders API.

- **Description:** Allows an authenticated user with a valid, unexpired session token to obtain a new token with a renewed expiration time.
- **Participants:** Customer
- **Resource(s):** Customer
- **Emitted Events:** Customer.SessionRefreshed
- **Characteristics:** unsafe, sync
- **Benefit:** This improves security by allowing for shorter token lifecycles while maintaining a seamless user experience, preventing the user from being logged out unexpectedly during an active session.

2. Add an Operation to Retrieve a Customer's Cart

Finding: The sequence diagrams assume a user always knows their cartId. However, a returning registered customer who added items to their cart in a previous session and then logged out would have no way to retrieve their persistent cart. The current `getCart(cartId)` operation in the Orders API requires the ID, which the user would not have.

.... truncated for space...



Add item 1. Also add support for removing an item from the cart and emptying the cart. Remove the authenticate operation as we will use OAuth2 framework and an IdP to manage the authentication flow. Update diagrams.

30

Finally, perform a check of everything to see if something is missing. This might happen if you opted to add or remove an API operation after your diagrams were initially generated.

31

If needed, incorporate any recommendations provided by the model. In this example, it was discovered that the shopping cart wasn't initially created for the shopper before the first item was added. Discuss this requirement and user flow with your SMEs, then adjust if needed.

32

Use prompts to make necessary changes.



Design

Starting the Design Phase

Transform the API profiles into high-level API design artifacts.

Coach's Notes

Human Prompt Engineer

The Define phase is now complete. Let's start the Design phase. Here are the rules and restrictions for this effort:

- * The Align and Define phases have been completed already
- * Until I tell you otherwise, let's stay in the Design phase and not blend any activities from the other phases of ADDR
- * By staying only in the Design phase, we will be required to produce a high-level API design
- * For this design, let's assume we will use RESTful practices and leverage the HTTP protocol, including HTTP methods, response codes, and headers, appropriately
- * We will not consider the Design phase complete until we have the high-level API design artifact complete for each API profile provided from the Define phase
- * Each high-level API design will be summarized in a table that contains the following columns: HTTP Method, Resource Path, Operation Name, Description, Request, Response (Success), and Response (Error)
 - * The Request and Response columns will contain details about input and output data related to query filters, identifiers, and resources referenced
 - * The Response (Error) column should include the proper 2xx success code based on the operation type and any 4xx family of HTTP response codes that the operation might produce
- * API operations have already been identified in the Define phase artifacts, which will be provided
- * Be sure to select the appropriate HTTP method based on the characteristics of each operation
- * When we have determined that we have completed the Design phase, you will package the final Design deliverable as a single, clean reference sheet so it's ready for hand-off to the Refine phase

Let's start by having you explain the phase, these artifacts, and why it is important so that we are clear.

33

With the Define phase complete, we have a better understanding of the APIs we need and the operations they will offer.

Next, we enter the Design phase. In this phase, we will apply our API design style guide to a high-level API design, based upon the API profiles we created previously.

We will prompt the model to transition us to the Design phase, constrain the work to only this phase, and have the model coach us on what is required.



Design

Starting the Design Phase

Transform the API profiles into high-level API design artifacts.

Coach's Notes



LLM Response

Excellent. Let's proceed to the Design phase. The goal of this phase is to translate the protocol-agnostic blueprint we created in the Define phase into a concrete, technology-specific design. We're moving from the abstract structure to the tangible details of how a developer will interact with our API over the network. In our case, we'll be applying RESTful principles using the HTTP protocol.

The High-Level API Design Artifact

The primary artifact we'll produce is a High-Level API Design for each of our three APIs (Catalog, Orders, and Customers). This document will serve as the technical contract for our API, specifying every detail an API consumer needs to know to interact with it successfully.

We will create a table for each API that details every operation. Here's a breakdown of the columns and what they represent:

- **HTTP Method:** We will select the appropriate HTTP verb (GET, POST, PUT, DELETE, etc.) that best matches the characteristics of the operation we defined earlier. For example, a safe operation will map to GET, while an idempotent state-changing operation will map to PUT or DELETE.
- **Resource Path:** This is the specific URI for the endpoint. We will follow RESTful conventions, using plural nouns for collections (e.g., /books) and path parameters for specific instances (e.g., /books/{bookId}).
- **Operation Name:** The familiar, business-friendly name from the Define phase (e.g., getBookDetails()). This links our technical design directly back to our business requirements.
- **Description:** A brief summary of the endpoint's purpose.
- **Request:** The data the consumer sends to the API, including any path parameters, query parameters for filtering/sorting, and the structure of the request body.
- **Response (Success):** The data and status code the API returns on a successful request. This includes the structure of the response body and the appropriate 2xx HTTP status code (e.g., 200 OK, 201 Created, 204 No Content).
- **Response (Error):** The potential 4xx family of HTTP status codes the operation might produce for client-side errors (e.g., 400 Bad Request, 404 Not Found).

Why This Phase is Crucial

This phase is critical because it creates the final, implementable specification for our API.

1. **Creates a Clear Contract:** This artifact removes all ambiguity. Developers consuming our API and the teams building it will have a precise, shared understanding of how it works. 🧑‍💻
2. **Enforces Consistency:** By systematically applying RESTful design patterns, we ensure our API is predictable, logical, and easy for developers to learn and use.
3. **Enables Parallel Development:** With this design finalized, teams can work in parallel. The backend team can build the API, frontend and mobile teams can develop against a mock server based on this contract, and technical writers can draft the official documentation. 📄

Let's begin by creating the high-level design for our first API.

34

The LLM demonstrates its understanding of the ADDR design phase by reminding us that we will transform API profile(s) from the Define phase into a high-level HTTP API design using REST-based practices.

Most models will generate HTTP details, including naming conventions and choice of URL formats, based on the data it was trained upon.

After we review the purpose of the phase, we will need to share our specific API design style preferences.



Design

Establishing Style Guidelines

Prompt the model to apply specific style guidelines to match the enterprise style guide.

Coach's Notes



For reference, attached is a list of style guide preferences, to help you with naming conventions, HTTP method selection, and URL path creation

1. General API Principles:

- * Design APIs to be intuitive, consistent, and interoperable within an open ecosystem.
- * Follow RESTful principles, ensuring each API is a well-defined resource with appropriate HTTP methods.
- * Default response format must be JSON, adhering to RFC8259 standards.

2. URL and Resource Design:

- * Base path structure: {business-domain}/{v#}/{resource-type}/{id}.
- * URLs must be human-readable, avoid application-specific names, and use hyphen-delimited business domains.
- * Major version (v#) must be included and follow a whole-number format (e.g., /v1/).
- * Resources should be nouns in plural form (except singletons) and use UUIDs as identifiers.

3. HTTP Methods and Functional Resources:

- * Use standard HTTP methods:
 - * GET: Retrieve resources.
 - * POST: Create resources or execute commands (Use a verb name for the function, e.g. search, but do not use \$ prefix for functions).
 - * PATCH: Partially update a resource.
 - * PUT: Full resource replacement (limited usage).
 - * DELETE: Remove a resource and return 204 No Content (no response body).
- * Functional endpoints (e.g., /resource/search, /resource/validate) must be invoked with POST.

4. Query Parameters & Filtering:

- * Use lowerCamelCase for query parameters.
- * Filters must support:
 - * AND conditions for multiple parameters.
 - * OR conditions using comma-separated values.
- * Reserved functional parameters start with _ (e.g., _include).
- * Sorting must be indicated using sort:
 - * Example: GET /v1/orders?sort=createdDate.

5. Response Handling and Standardization:

- * Use HTTP status codes appropriately:
 - * 200 OK: Successful retrieval of resources.
 - * 201 Created: New resource created; response must contain the full resource representation.
 - * 204 No Content: DELETE must return 204 No Content, not a message.
 - * 400 Bad Request: Invalid request.
 - * 404 Not Found: Resource does not exist.
 - * 409 Conflict: Business validation failure.
 - * 500 Internal Server Error: Unexpected issues.
- * POST, PUT, and PATCH responses must include the full resource representation rather than a success message.
- * JSON responses must avoid envelope-wrapping (e.g., {"status": "success", "data": {...}} is not allowed).

.... continued...

35

First, we need to provide a list of API design standards to ensure our API is consistent across other APIs in our portfolio. This step often removes the need for API linting or limits the errors our linters would otherwise generate if we designed our APIs manually. This is a huge time saving step.

Tip: Submit your API style guide to an LLM and request that it create a prompt that will direct the LLM when designing an API. This will save considerable time writing the prompt manually based upon your style guide preferences.



Design

Establishing Style Guidelines

Prompt the model to apply specific style guidelines to match the enterprise style guide.

Coach's Notes



6. Data Formatting & Serialization:

- * Dates must follow RFC-3339.
- * Use ISO 8601 for durations and intervals.
- * 64-bit integers exceeding JavaScript limits must be returned as strings.
- * Collections must be JSON arrays with each item containing a resourceId.

7. Security & Compliance:

- * Do not pass sensitive data via query parameters; use POST for searches involving PII.
- * Support OAuth 2.0 and API keys for authentication.
- * Implement CORS with appropriate access controls.

8. Pagination & Sorting (Offset-based Pagination):

- * Use offset to specify the starting position (zero-based index).
- * Use limit to define the number of instances to fetch.
- * Default values should be defined in the API specification.
- * Sorting must be indicated via sort. Example: GET /v1/orders?offset=20&limit=10&sort=createdDate
- * Retrieves 10 orders starting from the 21st record, sorted by createdDate.

9. Error Handling & Problem Details:

- * Use RFC 9457 Problem Details (formerly RC 7807) format for error responses.

prompt continues...



Understood. I have reviewed the REST API Style Guide and will adhere to its conventions as we proceed.

36

Once the model has understood the style guide, you are ready to proceed.



Design

Designing the APIs

Ask the LLM to generate a high-level design of each API using the style guidelines.

Coach's Notes



Let's begin the Design phase by creating the high-level API design for our first boundary: the Product Catalog API

37

Since most APIs will contain quite a few operations and resources, it is best to limit the design effort to the first API when you have a large scope of effort.



Product Catalog API Design

Description: Provides public access to discover, browse, and evaluate all books and bundles sold in JSON's Bookstore. It is the single source of truth for product information for both customers and external partners.

HTTP Method	Resource Path	Operation Name	Description	Request	Response (Success)	Response (Error)
POST	/product-catalog/v1/books/search	searchBooks()	Finds books matching specific query criteria.		Body: searchQuery, filters 200 OK: listOf(Book) 400: Bad Request	
GET	/product-catalog/v1/categories/{categoryId}/books	browseByCategory()	Retrieves a list of books within a given category.		Path: categoryId Query: sort, offset, limit 200 OK: listOf(Book) 404: Not Found	
GET	/product-catalog/v1/curated-lists/{listId}	getCuratedList()	Retrieves a specially curated list of books.	Path: listId Query: offset, limit	200 OK: listOf(Book or Bundle) 404: Not Found	
GET	/product-catalog/v1/books/{bookId}	getBookDetails()	Retrieves detailed information for a single book.		Path: bookId 200 OK: Book representation 404: Not Found	
GET	/product-catalog/v1/works/{workId}/reviews	getBookReviews()	Fetches all customer reviews for a book's work.	Path: workId Query: sort, offset, limit	200 OK: listOf(Review) 404: Not Found	
GET	/product-catalog/v1/books/{bookId}/recommendations	getRecommendations()	Retrieves a list of recommended books.	Path: bookId Query: offset, limit	200 OK: listOf(Book) 404: Not Found	
GET	/product-catalog/v1/inventory/{itemId}	getInventoryStatus()	Gets the availability status for a product.	Path: itemId	200 OK: Inventory representation 404: Not Found	

38

Review the generated API design in its tabular format first. Adjust your style guide prompt as needed to ensure you are fully compliant with your organization's API design style guide.

39

Once everything looks good, proceed to transform each API profile into a high-level API design.



Generate the high-level API design table for the next API



Design

Checking the Design Phase

Cross-check the artifacts against previous phase artifacts for any gaps or improvements.

Coach's Notes



Let's finalize this phase by cross-checking everything. Perform the following steps:

- Cross check every operation from the Define phase against the design tables to ensure nothing was dropped or altered in intent.
- Confirm each job story → operation → API design mapping is intact and still consumer aligned.
- Verify naming conventions, URI patterns, and HTTP method usage are consistent across all boundaries.
- Ensure error handling patterns are uniform (status codes, error payload structure — even if payload details are finalized in Refine).

40

The Design phase goes fast, as most of the effort of API design was focused in the Align and Define phases. The only thing left is to apply your API style guide and then cross-check that your API design reflects your previous artifacts.



Of course. Here is a final cross-check of the Design phase artifacts to ensure everything is consistent and complete.

1. Operation Coverage Check

I have cross-checked every operation identified in the Define phase against the final API design tables.

- Result:  Pass. All operations from the Product Catalog, Orders, and Customer Accounts API profiles have been successfully translated into a specific HTTP endpoint. No operations were dropped, and the intent of each operation has been preserved.

2. Job Story Alignment Check

I have traced the path from the initial job stories through the operations to the final API design.

- Result:  Pass. The API designs remain perfectly aligned with the consumer-centric goals of the original job stories. For example, JS2 (Complete a One-Time Purchase) is fully supported by a logical sequence of POST /carts, POST /carts/{cartId}/items, and POST /orders endpoints, creating a clear and intuitive workflow for the API consumer.

3. Consistency Verification

I have reviewed the naming conventions, URI patterns, and HTTP method usage across all three API designs.

- Result:  Pass. The design demonstrates strong consistency:
 - o Naming Conventions: All resource paths use hyphen-delimited business domains (e.g., /product-catalog, /orders-management).
 - o URI Patterns: The URI structure /{business-domain}/v1/{resource} is applied uniformly. Plural nouns are used for collections, and functional endpoints like /cancel and /search are used consistently.
 - o HTTP Method Usage: The selection of HTTP methods is consistent with the style guide and the operation's characteristics. GET is used for safe retrievals, POST is used for resource creation and functional commands, PATCH for partial updates, and DELETE for removals.

41

Make any adjustments as needed, based on your own review and the model's cross-check.



Refine

Starting the Refine Phase

Generate assets such as OpenAPI Specification documents, README docs, and diagrams.

Coach's Notes



Human Prompt Engineer

Let's proceed to the Refine Phase. Here are the rules and restrictions for this effort:

- The Align, Define, and Design phases have been completed already. I will attach those artifacts separately
- Until I tell you otherwise, let's stay in the Refine phase and not blend any activities from the other phases of ADDR
- By staying only in the Refine phase, we will be required to produce an OpenAPI Specification of each API and a README document that demonstrates HTTP request/response examples for each job story
- We will not consider the Refine phase complete until we have completed the artifacts for this phase
- When we have determined that we have completed the Refine phase, you will package the final Refine deliverable as a single, clean reference sheet so it's ready for hand off to the delivery team

Let's start by having you explain the phase, these artifacts, and why it is important so that we are clear. Since I have already gathered artifacts from the previous phases, I will provide those next.



Excellent. Let's move into the final phase of our process: the Refine phase. The goal of this phase is to take the high-level REST design we just completed and transform it into a formal, implementation-ready specification. This is where we finalize all the technical details, creating the definitive contract that developers will use to build, test, and consume the APIs.

Key Artifacts for the Refine Phase

We will produce two critical artifacts for each of our three APIs.

1. OpenAPI Specification

The OpenAPI Specification (OAS) is the primary technical artifact. It is a machine-readable YAML or JSON file that describes every single detail of the API in a standardized format. This includes:

- **Paths and Operations:** The exact URI paths and their corresponding HTTP methods (GET, POST, etc.).
- **Parameters:** Detailed definitions of all path, query, and header parameters.
- **Request/Response Bodies:** Precise JSON schemas that define the exact structure, data types, and validation rules (e.g., required fields, string patterns) for all data sent to and from the API.
- **Security Schemes:** Formal definitions of the security mechanisms, such as OAuth2.
- **Servers:** The base URLs for different environments (e.g., development, production).

The OAS is the single source of truth for the API's contract.

.... truncated for space...

42

We now enter the final phase of ADDR, the Refine phase. In this phase, we produce artifacts that help to obtain fast feedback. Doing so now allows us to make changes when the cost of change is still low.

43

Review the artifacts for the Refine phase based upon the coaching of the model.

In some cases, you may want to ignore some artifacts and generate others that your organization prefers to use.



Refine

Generating OpenAPI Documents

Using the work completed so far, generate a complete OpenAPI document for review.

Coach's Notes



Generate a fully compliant OpenAPI 3.1 specification that reflects the job stories and activity steps submitted previously and that adheres to the API design standards described above. Include required fields and examples for each property. Examples should be consistent across operations when possible, to demonstrate a basic workflow. Group operations under a tag and include tag meta data to properly document each group of operations. Start with the first API.

44

Here is the fully compliant OpenAPI 3.1 specification for the first API.

```
openapi: 3.1.0
info:
  title: Product Catalog API
  description: Provides public access to discover, browse, and evaluate all books and bundles sold in JSON's Bookstore.
  version: "1.0.0"
tags:
  - name: Product Catalog
    description: Discover and evaluate all products available for sale.
paths:
  /product-catalog/v1/books/search
post:
  tags:
    - Product Catalog
  summary: Search for Books
  operationId: searchBooks
  description: Finds books matching specific query criteria.
  requestBody:
    required: true
    content:
      application/json:
        schema:
          type: object
          properties:
            searchQuery:
              type: string
              example: "API Design"
          filters:
            type: object
            properties:
```

.... truncated for space...



45

Generating an OpenAPI document is often the most time-consuming step in any API design process. Since we have done the hard work up-front to produce our ADDR-based artifacts, LLMs can generate complete and thoughtful API documentation quickly using a single prompt.

Some models introduce minor OpenAPI errors that can be corrected via follow-up prompts or with a small, manual fix. Most models, however, will generate valid OpenAPI documents.



LLM-Generated Artifact: OpenAPI Documents

Example output generated by LLM.

Product Catalog API

1.0.0OAS 3.1

Provides public access to discover, browse, and evaluate all books and bundles sold in JSON's Bookstore.

Servers

https://api.jsonbookstore.com

Product Catalog

Discover and evaluate all products available for sale.

POST

/product-catalog/v1/books/search

Search for Books

GET

/product-catalog/v1/categories/{categoryId}/books

Browse by Category

GET

/product-catalog/v1/curated-lists/{listId}

Get a Curated List

GET

/product-catalog/v1/books/{bookId}

Get Book Details

GET

/product-catalog/v1/works/{workId}/reviews

Get Book Reviews

GET

/product-catalog/v1/books/{bookId}/recommendations

Get Recommendations

GET

/product-catalog/v1/inventory/{itemId}

Get Inventory Status

Schemas

Book >

Expand all

object

Bundle >

Expand all

object

Review >

Expand all

object

Inventory >

Expand all

object

Error >

Expand all

object

Orders API

1.0.0OAS 3.1

Manages the entire transactional workflow, including shopping cart management, checkout processing, and post-purchase order management for all customers.

Servers

https://api.jsonbookstore.com

Orders & Carts

Operations for creating and managing shopping carts and customer orders.

POST

/orders-management/v1/carts

Create a Cart

GET

/orders-management/v1/carts/{cartId}

Get a Cart

GET

/orders-management/v1/customers/{customerId}/cart

Get Customer's Cart

POST

/orders-management/v1/carts/{cartId}/items

Add an Item to a Cart

PATCH

/orders-management/v1/carts/{cartId}/items/{itemId}

Update a Cart Item

DELETE

/orders-management/v1/carts/{cartId}/items/{itemId}

Remove a Cart Item

POST

/orders-management/v1/carts/{cartId}/empty

Empty a Cart

POST

/orders-management/v1/carts/{cartId}/apply-discount

Apply a Discount Code

POST

/orders-management/v1/orders

Create an Order

GET

/orders-management/v1/orders/{orderId}

Get an Order

GET

/orders-management/v1/customers/{customerId}/orders

List a Customer's Orders

POST

/orders-management/v1/orders/{orderId}/cancel

Cancel an Order

Schemas

Cart >

Expand all

object

CartItem >

Expand all

object

Order >

Expand all

object

Generated documentation based upon the LLM-generated OpenAPI document



Refine

Generating README Documentation

Demonstrate the API usage using a README doc, the basis for a “Getting Started” guide.



Let's produce README-based request/response examples using Markdown for each of the job stories that demonstrate the use of the API design.

JS1: Discover Relevant Books

A customer wants to find a book about "API Design" and check its details and reviews.

1. Search for the book

The customer searches for books with "API Design" in the title.

Request:

```
curl -X POST 'https://api.jsonbooksstore.com/product-catalog/v1/books/search' \
-H 'Content-Type: application/json' \
-d '{
  "searchQuery": "API Design"
}'
```

Response 200 OK:

```
JSON
[
  {
    "bookId": "a1b2c3d4-e5f6-7890-1234-567890abcdef",
    "worldId": "b2c3d4e5-f6a7-8901-2345-67890abcdef0",
    "title": "Designing Web APIs",
    "author": "Jane Doe",
    "format": "Paperback",
    "price": 24.99
  }
]
```

2. Get book details

The customer retrieves the full details for the book they found.

.... truncated for space...



Coach's Notes

46

OpenAPI documentation is a helpful reference, but API consumers also need to understand how they will use your APIs to deliver on desired outcomes captured in your job stories.

47

LLMs equipped with the artifacts previously generated can quickly generate README-style documentation that guides the reader step-by-step in using your API to deliver those desired outcomes.

Review the generated documentation then address any gaps or improvements you identify.



Refine

LLM-Generated Artifact: README Documentation

Example output generated by LLM.

JSON's Bookstore API - README and Examples

This document provides practical examples of how to use the JSON's Bookstore APIs to perform key tasks. Each example corresponds to a specific user goal (Job Story).

JS1: Discover Relevant Books

A customer wants to find a book about "API Design" and check its details and reviews.

1. Search for the book

The customer searches for books with "API Design" in the title.

Request:

```
Bash
curl -X POST 'https://api.jsonbookstore.com/product-catalog/v1/books/search' \
-H 'Content-Type: application/json' \
-d '{
  "query": "API Design"
}'
```

Response 200 OK:

```
JSON
{
  {
    "id": "alb2c3d4-e5f6-7890-1234-567890abcdef",
    "id": "b2c3d4e5-f6a7-8901-2345-67890abcdef0",
    "title": "Designing Web APIs",
    "author": "Jane Doe",
    "format": "Paperback",
    "price": 24.99
  }
}
```

2. Get book details

The customer retrieves the full details for the book they found.

Request:

```
Bash
```

```
curl -X GET 'https://api.jsonbookstore.com/product-catalog/v1/books/alb2c3d4-e5f6-7890-1234-567890abcdef'
```

Response 200 OK:

```
JSON
{
  "id": "alb2c3d4-e5f6-7890-1234-567890abcdef",
  "id": "b2c3d4e5-f6a7-8901-2345-67890abcdef0",
  "title": "Designing Web APIs",
  "author": "Jane Doe",
  "format": "Paperback",
  "price": 24.99,
  "language": "English",
  "synopsis": "An in-depth guide to designing modern APIs...",
  "image": "https://api.jsonbookstore.com/images/alb2c3d4.jpg"
}
```

3. View book reviews

The customer decides to check the reviews for the book.

Request:

```
Bash
curl -X GET 'https://api.jsonbookstore.com/product-catalog/v1/works/b2c3d4e5-f6a7-8901-2345-67890abcdef0/reviews'
```

Response 200 OK:

```
JSON
{
  {
    "id": "f1e2d3c4-b5a6-7890-1234-567890abcdef",
    "id": "b2c3d4e5-f6a7-8901-2345-67890abcdef0",
    "author": "John S.",
    "rating": 5,
    "comment": "A must-read for any developer!"
  }
}
```

Example of a generated README doc that shows how the API will be used to realize job story JS1



Refine

Checking the Refine Phase

Cross-check the Refine artifacts against all previous artifacts.

Coach's Notes



Here is the final cross-check of the README examples against the API design.



Final Artifact Cross-Check

I have reviewed the README examples against the final API designs and OpenAPI specifications to ensure complete alignment.

- Operations Check: ☒ Pass. Every curl command in the README maps directly to an operation defined in the API design tables and OpenAPI files. No new or undocumented operations were introduced.
- Resources Check: ☒ Pass. The JSON payloads in the request and response examples use the exact resource structures (Book, Cart, Order, Customer, etc.) and properties defined in the final resource models. No new or undefined properties were added.
- Workflow Logic Check: ☒ Pass. The sequence of API calls shown for each job story is logical and accurately reflects the intended use of the APIs. For example, the guest checkout flow correctly creates a cart before adding an item and creating the order, using consistent identifiers across steps.

Conclusion

The README examples are a faithful and accurate representation of the final API design. All artifacts are in alignment. The Refine phase is complete.

48

Use the LLM's cross-check capabilities to spot any differences between your changes and previous artifacts.

49

Use prompts to make any final changes. You may want to ask the model to generate a fresh set of artifacts that reflect any changes you made.

50

Congratulations! You now have a thoughtful, well-designed API that focuses on the needs of users rather than purely exposing raw data. Use this or a fresh LLM chat session to make any further improvements as needed.



4 Choosing Your AI Assistant

LLM Capability Matrix

Select the model that best fits your need

- Ratings and findings
- Capability matrix (by LLM model)
- Model evaluation methodology

Anthropic Claude Sonnet 4.5

Findings:

- Wasn't trained or aware of ADDR.
- However, the model was able to be trained with ADDR website to develop basic awareness.
- Had to redirect the model at times with prompts.
- Didn't offer a full coaching process but satisfactory experience considering the on-the-fly learning.
- Artifact generation was slower than most models but was error-free and complete.



Microsoft Copilot with Smart GPT-5

Findings:

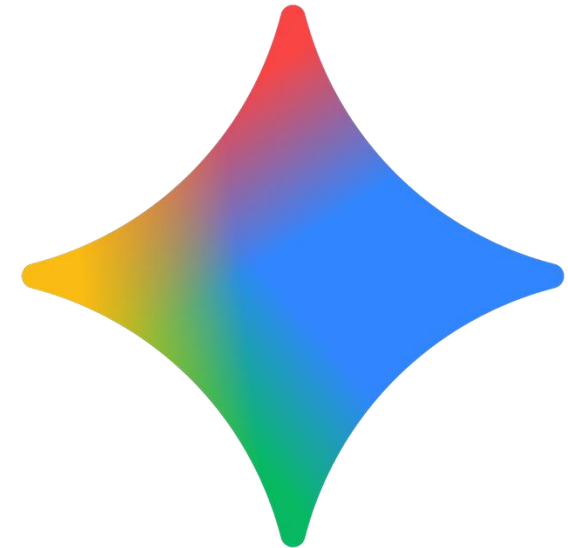
- Good domain modeling capabilities.
- Was unable to conduct the full ADDR process in a single chat session.
- Some artifacts had syntax errors that needed to be fixed.
- Recommended additional artifacts and actions along the way that could be useful.



Google Gemini 2.5 Pro

Findings:

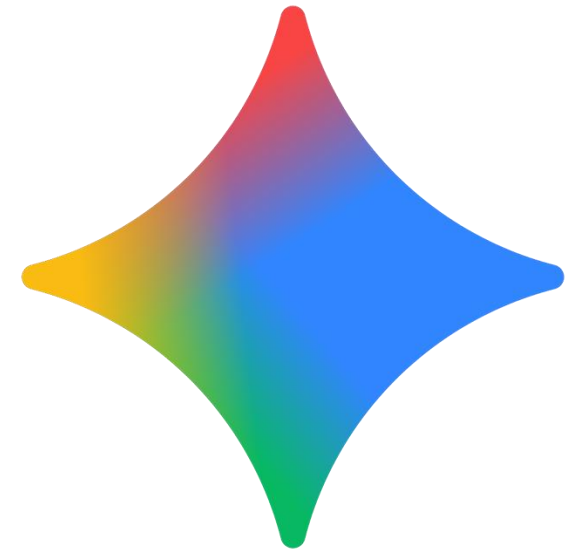
- Excellent domain modeling capabilities.
- Able to conduct the full ADDR process in a single chat session with minimal prompt guidance required.
- Very fast responses.
- Artifacts were complete and error-free.
- Lacked some gap analysis capabilities regarding the API design.



Google Gemini 3.0 Thinking

Findings:

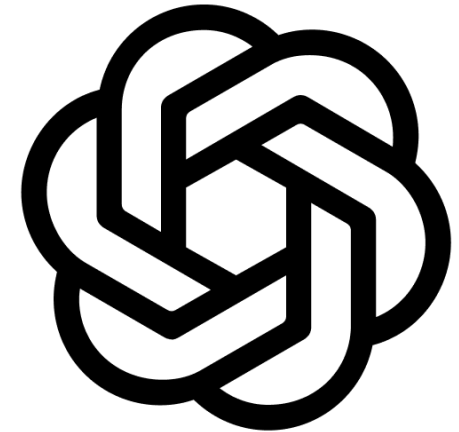
- Excellent domain modeling capabilities.
- Able to conduct the full ADDR process in a single chat session with minimal prompt guidance required.
- Good at coaching through the full process.
- Slower than Gemini 2.5 Pro by 2-3x.
- Better at identifying gaps than Gemini 2.5 Pro.



OpenAI GPT-5.1 Auto

Findings:

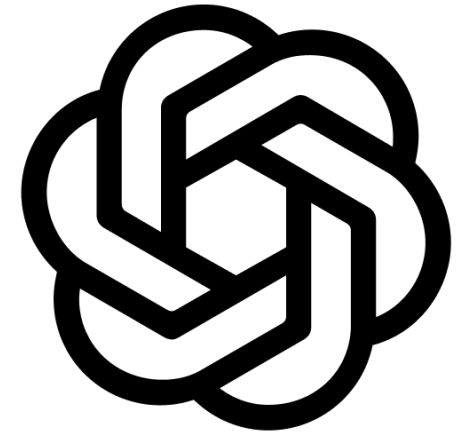
- Was familiar with ADDR and capable of performing all tasks.
- Better job story creation experience that avoided too many, fine-grained job stories.
- Good at identifying gaps throughout the process.
- Struggled with smaller boundaries for each API, which required additional prompting to better align with job story outcomes.



OpenAI GPT-5 Pro

Findings:

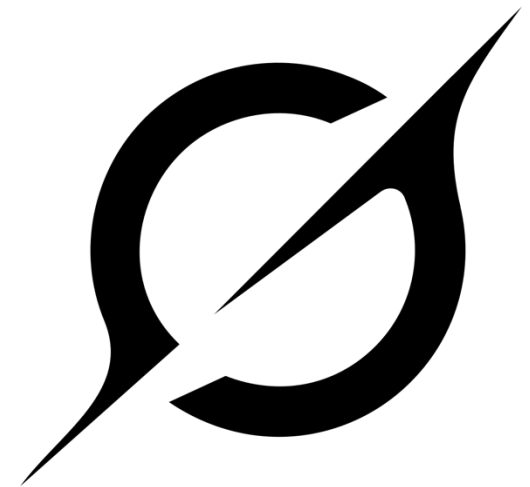
- Very slow with no additional insights offered.
- While the requirements Q&A prompting resulted in fewer questions asked, the time to process prompts outweighs the savings.
- Unable to follow strict prompt instructions when outlining user journeys.
- Use only for deep research on business requirements.
- NOT RECOMMENDED for API design coaching.



Grok 4 Expert

Findings:

- Comprehensive business requirements assessment.
- Understands ADDR but lacked coaching guidance.
- Some generated artifacts were not valid.
- Limited screen width given to table outputs, making it difficult to work with the model to design the API (UX issue).



Note: At the time of this report, Grok 4.1 is in beta. We will review this model in a future report due to frequent changes to the model during its beta.

LLM Capability Findings - Details



	Anthropic Claude Sonnet 4.5	Microsoft Copilot Smart GPT-5	Google Gemini 2.5 Pro	Google Gemini 3.0 Thinking	OpenAI GPT-5.1 Auto	Grok 4 Expert
ADDR Awareness						
Model is aware of ADDR process	No	Yes	Yes	Yes	Yes	Yes
Model explains ADDR phases and artifacts	Yes ³	Yes	Yes	Yes	Yes	Yes
Model engages in a coached ADDR session	No	Yes	Yes	Yes	Yes	No
Model transitions to each ADDR phase without starting a new chat session	Yes	No	Yes	Yes	Yes	Yes
Business Requirements						
Model asks clarifying questions for business requirements	Yes	Yes	Yes	Yes	Yes	Yes
Model recommends improvements to avoid rework and breaking API changes	Yes	No	Yes	Yes	Yes	Yes

¹ Some intentional gaps were missed by the model and had to be prompted by the API designer

² Model may introduce minor errors that have to be corrected, but overall produces a complete and valid document

³ Not built in. Model had to search the web to understand the concept

⁴ Not by default. Additional human prompt guidance

LLM Capability Findings - Details(con't)



	Anthropic Claude Sonnet 4.5	Microsoft Copilot Smart GPT-5	Google Gemini 2.5 Pro	Google Gemini 3.0 Thinking	OpenAI GPT-5.1 Auto	Grok Expert
Align Phase						
Model is aware of job story concepts	Yes	Yes	Yes	Yes	Yes	Yes
Model generates unifying job stories from requirements in the job story format	No ⁴	Yes	Yes	Yes	Yes	Yes
Model evaluates job stories and identifies gaps	Yes	Yes	Yes	Yes	Yes	Yes
Model identifies follow-up questions based on job stories	Yes	Yes	Yes	Yes	Yes	Yes
Model captures activity steps for each job story	No ⁴	Yes	Yes	Yes	Yes	Yes
Model requests clarifications of activity steps	Yes	Yes	Yes	Yes	Yes	Yes
Model evaluates and identifies gaps in activity steps	Yes	Yes	Yes	Yes	Yes	Yes
Model determines if sufficient information exists to proceed to the Define Phase	No	Yes	Yes	Yes	Yes	No

¹ Some intentional gaps were missed by the model and had to be prompted by the API designer

² Model may introduce minor errors that have to be corrected, but overall produces a complete and valid document

³ Not built in. Model had to search the web to understand the concept

⁴ Not by default. Additional human prompt guidance

LLM Capability Findings - Details(con't)



	Anthropic Claude Sonnet 4.5	Microsoft Copilot Smart GPT-5	Google Gemini 2.5 Pro	Google Gemini 3.0 Thinking	OpenAI GPT-5.1 Auto	Grok Expert
Define Phase						
Model recommends API resources given Align artifacts	Yes	Yes	Yes	Yes	Yes	Yes
Model recommends API operations given Align artifacts	Yes	Yes	Yes	Yes	Yes	Yes
Model identifies gaps with follow-up questions for captured API operations	Yes	Yes	Yes ¹	Yes	Yes	Yes
Model recommends API boundaries that ensure high cohesion and loose coupling	No ⁴	Yes	Yes	Yes	Yes	Yes
Model produces a model sequence diagram or sequence script for each job story	Yes	Yes	Yes	Yes	Yes	Yes
Model produces an error-free model sequence diagram or sequence script for each job story	Yes	Yes	Yes	Yes	Yes	Yes
Model evaluates API operations for key characteristics (safe, unsafe, idempotence)	Yes	Yes ²	Yes	Yes	Yes	Yes
Model determines if sufficient information exists to proceed to the Design Phase	No	Yes ²	Yes	Yes	Yes	No

¹ Some intentional gaps were missed by the model and had to be prompted by the API designer

² Model may introduce minor errors that have to be corrected, but overall produces a complete and valid document

³ Not built in. Model had to search the web to understand the concept

⁴ Not by default. Additional human prompt guidance

LLM Capability Findings - Details(con't)



	Anthropic Claude Sonnet 4.5	Microsoft Copilot Smart GPT-5	Google Gemini 2.5 Pro	Google Gemini 3.0 Thinking	OpenAI GPT-5.1 Auto	Grok Expert
Design Phase						
Model transforms an API style guide into prompt format	Yes	Yes	Yes	Yes	Yes	Yes
Model generates a REST-based API design based upon the Define Artifacts	Yes	Yes	Yes	Yes	Yes	Yes
Model generates a REST-based API design that complies with the API design rules	Yes	Yes	Yes ¹	Yes	Yes	Yes
Model generates a REST-based API design that applies each operation's Define characteristics	Yes	Yes	Yes	Yes	Yes	Yes
Model determines if sufficient information exists to proceed to the Refine Phase	No	No ⁴	Yes	Yes	Yes	No

¹ Some intentional gaps were missed by the model and had to be prompted by the API designer

² Model may introduce minor errors that have to be corrected, but overall produces a complete and valid document

³ Not built in. Model had to search the web to understand the concept

⁴ Not by default. Additional human prompt guidance

LLM Capability Findings - Details(con't)



	Anthropic Claude Sonnet 4.5	Microsoft Copilot Smart GPT-5	Google Gemini 2.5 Pro	Google Gemini 3.0 Thinking	OpenAI GPT-5.1 Auto	Grok Expert
Refine Phase						
Model generates an OpenAPI Specification (v3.0) for the current API design	Yes	Yes	Yes	Yes	Yes	Yes
Model generates a valid OpenAPI Specification (v3.0) for the current API design	Yes	Yes ²	Yes	Yes ²	Yes ²	Yes ²
Model generates an OpenAPI Specification (v3.1) for the current API design	Yes	Yes	Yes	Yes	Yes	Yes
Model generates a valid OpenAPI Specification (v3.1) for the current API design	Yes	Yes ²	Yes	Yes ²	Yes ²	Yes ²
Model produces a design sequence diagram or sequence script for each job story	Yes	Yes	Yes	Yes	Yes	Yes
Model produces a valid design sequence diagram or sequence script for each job story	Yes	No	Yes	Yes	Yes	Yes
Model produces a README-style document showing request/response sets for each job story	Yes	Yes	Yes	Yes	Yes	Yes
Model produces a valid Postman Collection showing request/response sets for each job story	Yes	Yes	Yes	Yes	Yes	No

¹ Some intentional gaps were missed by the model and had to be prompted by the API designer

² Model may introduce minor errors that have to be corrected, but overall produces a complete and valid document

³ Not built in. Model had to search the web to understand the concept

⁴ Not by default. Additional human prompt guidance

Model Evaluation Approach

How each model was evaluated

Publicly Hosted Models. We focused on publicly-hosted production-ready (i.e., non-beta) models for this version of the report. This ensures that models are readily available and likely adopted by the organization. Pro or thinking models were selected whenever possible to balance speed with the necessary skills for thoughtful API design.

ADDR awareness and coaching. We evaluated models on their existing awareness of the ADDR process. In the case of a model that lacks awareness, we evaluated if they could learn ADDR for the specific session. We also evaluated if the model can coach the designer through the ADDR process.

Artifact Creation. Finally, we needed to ensure that artifacts could be generated by the model to avoid the additional step of human artifact creation after prompting is complete. This included the creation of OpenAPI, Mermaid.js diagrams, and README-style documentation for fast feedback loops.

Try it Yourself



Additional resources to get started

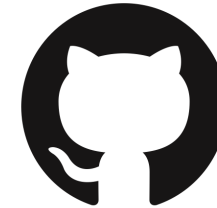
- About the Book “Principles of Web API Design”
- Learn ADDR at your own pace
- GitHub repository with starter prompts

Additional Resources

Learn more and accelerate your API design efforts

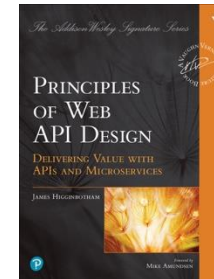
Prompt Starter Pack

Get started with starter prompts, guiderails, and tips in the GitHub Repo. Leverage these starter prompts to accelerate your AI-assisted API design coaching sessions.



Buy the Book

Purchase "Principles of Web API Design" to learn more about the ADDR process. The book brings together principles and processes to help you succeed across the entire API design lifecycle using REST-based, GraphQL, and gRPC-based APIs.



Take the Course

Gain hands-on experience with the instructor-led course by the creator of the ADDR process, James Higginbotham. Receive feedback from the author and receive a certificate of completion toward your API design coach certification.



About the Author

James Higginbotham is an API and AI coach, with experience in API strategy and execution, software architecture, and assisting teams in delivering API platforms and products.

As the founder of [LaunchAny](#), James guides organizations through their AI and digital transformation journey, ensuring alignment between business and technology through product-based thinking to deliver a great customer experience.

James has published several titles, including his latest book, "[Principles of Web API Design: Delivering Value with APIs and Microservices](#)", published by Addison-Wesley.

He also offers training courses at [APICoach.io](#).

